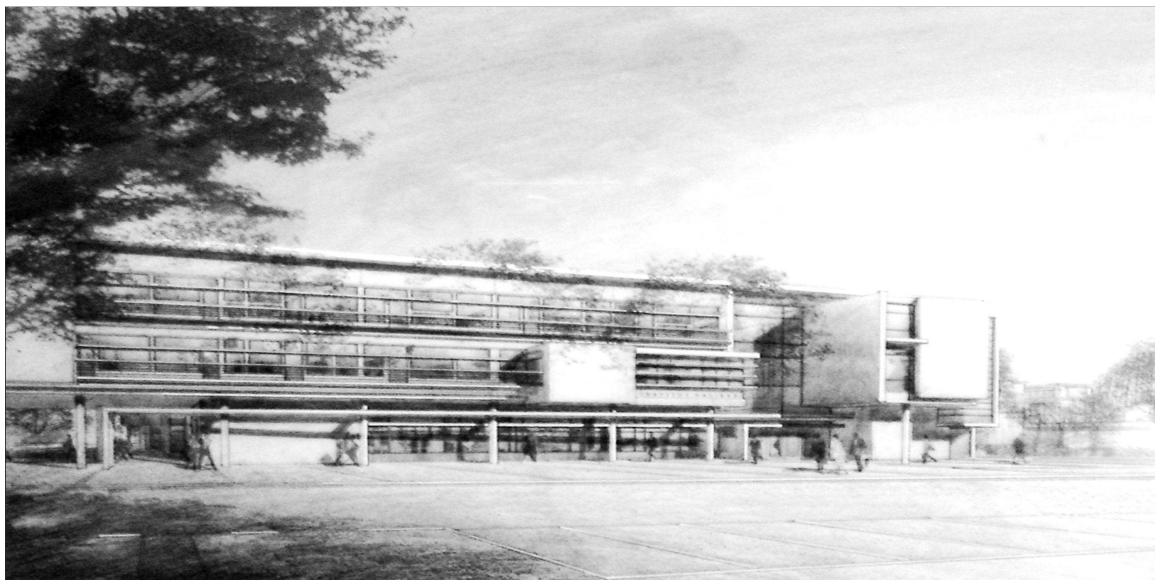




Licence SPI 2^{ème} année

MODULE MICROCONTRÔLEURS
ARCHITECTURE DES SYSTEMES
UTILISANT DES MICROPROCESSEURS

SOMMAIRE GENERAL

I. INTRODUCTION GENERALE	04
II. LES ASPECTS MATERIELS	07
III. A L'INTERIEUR DU MICROPROCESSEUR	48
IV. LES ASPECTS LOGICIELS	74
V. QUELQUES CONSEILS PRATIQUES ET UTILES	82

THIEBAUT Bernard

AVERTISSEMENT

Contrairement à l'électronique **analogique** où l'aspect formalisation mathématique était **important et indispensable**, en électronique **numérique** ce qui compte avant tout c'est la **logique** de raisonnement.

L'outil mathématique à mettre en œuvre ici se résume à des connaissances sur **l'algèbre de Boole**, sur les **fonctions combinatoires de base** (opérateurs logiques comme AND, OR, NAND, NOR, XOR ou la simple inversion logique), sur les **fonctions séquentielles** (mémorisation dans un registre, comptage, décalage ...) et sur **le codage de l'information** (binaire, hexadécimal ou ASCII). Certains de ces éléments ont été vus en Licence 1 dans le cours d'électronique numérique, d'autres seront fournis au fur et à mesure des besoins.

Ainsi, à mon sens, une des principales difficultés dans le domaine des architectures de systèmes à base de microprocesseurs ne vient pas des mathématiques à mettre en œuvre mais plutôt de la **quantité d'informations à intégrer** sachant que toutes ces informations prises **individuellement sont assez simples à comprendre**, pourvu qu'on soit un tant soit peu logique.

De plus, dans cette discipline, beaucoup d'éléments seront fournis par les **documentations constructeurs**. Vu le nombre de composants électroniques potentiels à mettre en œuvre, on ne peut bien sûr pas tout savoir. Par contre, il est indispensable de savoir bien lire et bien utiliser une documentation, qui, le plus souvent sera en **Anglais**. Un des objectifs de ce cours sera justement d'apprendre à utiliser correctement les documentations et à les mettre en perspective avec les éléments de base, qui eux doivent être connus.

Le module **microcontrôleur** sera ainsi divisé en deux parties :

- la première, correspondant à ce polycopié correspondra à des généralités concernant les systèmes architecturés autour d'un microprocesseur. On y présentera les différents éléments entrant dans la constitution de ce genre de systèmes et comment les associer entre eux pour assurer un fonctionnement cohérent de l'ensemble. On se familiarisera ainsi avec le **vocabulaire technique** en présentant des notions théoriques fondamentales, notions que l'on retrouvera dans la plupart des systèmes existants. Ces connaissances de base nous permettront ensuite de passer à la deuxième partie.

- on étudiera ensuite un composant particulier : le microcontrôleur **HC11** de **Motorola** sur lequel on mettra ainsi en pratique, et sur un cas concret, les notions générales et théoriques vues dans la première partie. Des séries de Travaux Pratiques permettront dans un premier temps d'apprendre à programmer le HC11 puis ensuite de l'interfacer avec un environnement extérieur.

Le présent document polycopié ne doit pas être considéré comme une présentation intégrale du cours de Monsieur THIEBAUT.

Il ne dispense en aucune façon de la présence aux cours.

Il est destiné à fournir aux étudiants intéressés un support permettant de traiter les points les plus importants.

Comme dans toute discipline " fortement " technique, il est vivement conseillé aux étudiants de compléter sa lecture par l'utilisation d'ouvrages plus importants et plus détaillés.

SOMMAIRE

I. INTRODUCTION GENERALE.....	04
II. LES ASPECTS MATERIELS.....	07
II.1. LES PIECES DU " PUZZLE ".....	07
II.1.1. LE MICROPROCESSEUR	07
II.1.2. LA MEMOIRE.....	07
II.1.3. L'EXTERIEUR (OU ENVIRONNEMENT)	22
II.2. CONSTRUCTION DU " PUZZLE "	30
III. A L'INTERIEUR DU MICROPROCESSEUR	48
III.1. LE PROGRAM COUNTER ET LE SEQUENCMENT	49
III.2. LE CYCLE INSTRUCTION	50
III.3. LE STATUS REGISTER	57
III.4. LE STACK POINTER	63
III.5. LES AUTRES REGISTRES	71
III.6. PROPOSITION D'ARCHITECTURE INTERNE (SIMPLIFIEE)	72
III.7. SYNTHESE : LE MODELE DE PROGRAMMATION	73
IV. LES ASPECTS LOGICIELS	74
IV.1. LE LANGAGE MACHINE	74
IV.2. UNE AMELIORATION : LE LANGAGE ASSEMBLEUR	75
IV.3. LE DEVELOPPEMENT D'UNE APPLICATION	78
IV.3.1. ANALYSE	79
IV.3.2. ECRITURE DU SOURCE	79
IV.3.3. COMPILATION ET LINKAGE DU SOURCE	79
IV.3.4. TELECHARGEMENT SUR LA CARTE APPLICATIVE	80
IV.3.5. TESTS ET MISE AU POINT	80
IV.3.6. IMPLANTATION EN ROM	80
V. QUELQUES CONSEILS PRATIQUES ET UTILES	82

I. INTRODUCTION GENERALE

Avant de rentrer plus en détail dans le cœur d'un **microprocesseur**, d'énumérer les différentes propriétés des **mémoires vives** ou les caractéristiques des principaux **circuits périphériques** que l'on rencontre le plus fréquemment, il est à mon avis important de se faire une idée sur les grands principes de fonctionnement des systèmes architecturés autour d'un microprocesseur (et par extension autour d'un **microcontrôleur**).

L'objectif de ce cours est donc double :

- définir les **notions fondamentales**, les points clefs et le **vocabulaire de base** "de la profession ",
- avoir une **idée globale** sur la logique du fonctionnement en faisant une approche plutôt **fonctionnelle** et non orientée vers tel ou tel composant spécifique.

On retrouvera ensuite la plupart de ces notions de base quand on étudiera un composant particulier, par exemple quand on abordera le microcontrôleur HC11.

L'approche de présentation qui a été choisie est également différente de celle qui dirait par exemple :

« Une architecture est constituée par :

- ...
- ...
- ... »

en donnant, de but en blanc, une liste de composants et de schémas, comme celui de la figure II.16., plus ou moins simples et sortis on ne sait d'où.

Je préfère poser la question suivante :

« De quoi a t'on besoin et comment associer ces composants de base pour que ça marche ? ».

C'est à mon avis une démarche plus proche de celle, initiale, des premiers concepteurs de systèmes informatiques, et qui permettra au final d'aboutir, un peu par déduction au schéma de la figure II.16., mais en ayant présenté auparavant ses différents éléments et la façon de les associer.

Pour cette raison, les composants que nous utiliserons ici sont **purement fictifs**, mais ils mettent en évidence un **nombre minimal d'éléments** que l'on trouvera toujours sur les architectures et les circuits intégrés **réels**.

Rassurez-vous, comme nous l'avons déjà dit plus haut, nous verrons des composants réels dans la deuxième partie de ce cours. Patience ...

Nous essaierons toutefois, autant que faire se peut, et notamment en TD de donner quelques exemples/exercices issus de cas réels pour concrétiser cette présentation.

Si cette partie est bien comprise, vous aurez plus de facilités pour vous adapter aux nombreux cas concrets et pour vous construire, petit à petit et de façon cohérente, une représentation mentale de plus en plus claire sur ce vaste domaine :

- différents constructeurs de microprocesseurs (Motorola, Intel, Texas, Analog Devices, Toshiba ...), de microcontrôleurs et/ou de processeurs de traitement numérique du signal,
- multitude de circuits intégrés d'interfaçage avec l'environnement,
- langages de programmation spécifiques comme l'assembleur ou plus généralistes comme le C/C++,
- outils de développement propres à chaque microprocesseur / microcontrôleur,
- applications diverses et variées ...

Comme je l'ai déjà dit, j'aime souvent faire la remarque suivante :

« Dans ce domaine, les idées de base sont simples et facilement compréhensibles (pas ou très peu de mathématiques compliquées et abstraites).

Par contre, la difficulté vient de la multitude de ces choses simples, souvent interdépendantes ou imbriquées les unes dans les autres »

Le souci principal qui a amené les concepteurs à développer ce type de systèmes résulte de la constatation suivante :

- avec les composants de base vus en Licence 1 (portes logiques, décodeurs, comparateurs, bascules, compteurs ...), si nous voulons développer une application particulière (par exemple un système de digicode comme on en trouve à l'entrée de la plupart des immeubles, ou tout autre système orienté vers un but précis), nous allons associer ces différents composants de base, bref différents circuits intégrés pour répondre au cahier de charges de cette application.

Nous obtiendrons ainsi une « carte électronique **spécifique** » " pour cette application.

Mais pouvons-nous utiliser cette carte pour une autre application ?

Non, bien sûr, et nous devons à chaque fois concevoir une nouvelle carte pour chaque nouvelle application.

C'est ce qu'on appelle la **LOGIQUE CÂBLEE**.

- avec les nouveaux composants que nous allons présenter dans cette partie, nous allons définir, concevoir et développer une architecture (plus ou moins standard), capable de s'adapter à un nombre quasiment infini d'applications, toutes différentes les unes des autres.

Mais à quel prix ? Faudra-t'il chaque fois " casser " cette architecture électronique ?

Non, on associera à chaque application nouvelle un **PROGRAMME** différent et spécifique.

C'est ce qu'on appelle la **LOGIQUE PROGRAMMEE**.

Remarque générale sur ce polycopié :

De façon très générale, les termes en **LETTRES MAJUSCULES** de ce polycopié indiqueront les « mots clés » importants du vocabulaire (Anglais ou Français) propres à ce domaine.

II. LES ASPECTS MATERIELS

Ils sont le plus souvent regroupés sous le terme générique de **HARD**, contraction de l'américain **HARDware**.

Indiquons tout de suite que l'autre grand domaine, abordé dans la quatrième partie est connu sous le terme de **SOFT**, contraction de **SOFTware**. (et correspondant aux programmes, aux logiciels).

II.1. LES PIECES DU " PUZZLE "

Les principaux éléments qui vont intervenir dans notre construction standard seront :

- LE **MICROPROCESSEUR**,
- LA **MEMOIRE**,
- LES **CIRCUITS PERIPHERIQUES PROGRAMMABLES**.

Nous allons ici présenter le rôle de chacune de ces « pièces ».

II.1.1. LE MICROPROCESSEUR

Cœur du système, c'est lui qui travaille (on dira **EXECUTE** les différentes tâches (appelées **INSTRUCTIONS ELEMENTAIRES**) qui lui sont demandées). Et pour le moment nous n'en dirons pas plus, si ce n'est qu'il est aussi parfois nommé **C.P.U.** (pour **CENTRAL PROCESS UNIT**) ou **U.C.** (pour **UNITE CENTRALE**).

II.1.2. LA MEMOIRE

C'est elle qui contiendra la liste des tâches (on dit aussi la liste des instructions à exécuter) que le microprocesseur aura à effectuer lorsqu'un programme se déroulera.

Un microprocesseur est un excellent exécutant mais il faut lui dire à tout instant quoi faire. Ce découplage, entre le microprocesseur et la mémoire correspond à une architecture type **Von Neumann** du nom du mathématicien et physicien américain qui l'a proposée la première fois.

Ainsi, c'est un des premiers rôles de la mémoire : **stocker les différentes instructions élémentaires** (comme faire une opération arithmétique ou logique) qui seront ensuite exécutées **par** et **dans** le microprocesseur.

Mais on conçoit bien que cela n'est pas suffisant. En effet, il est également important pour le microprocesseur de savoir sur quoi il doit travailler (il sait par exemple qu'il doit faire une addition mais avec quels opérandes ?) et où il doit ranger le résultat de son travail (dans l'exemple précédent la somme qu'il vient d'effectuer).

Cette première partition entre :

- ce que le microprocesseur doit faire et ...
- ce sur quoi il doit le faire et où il doit ranger ses résultats,

conduit naturellement à distinguer deux types de mémoire :

- la **R.O.M.** (de **READ ONLY MEMORY** ou **MEMOIRE A LECTURE SEULE**) contiendra le **PROGRAMME**, c'est-à-dire la liste des **INSTRUCTIONS** à effectuer.

Elle ne peut être que lue, ce qui à priori semble normal (tout au moins dans les cas simples qui nous intéressent ici) pour éviter que le microprocesseur ne modifie son propre travail.

Elle possède également la propriété fondamentale de conserver son contenu même lorsqu'elle n'est plus alimentée, ce qui est normal aussi car cela permet de conserver les instructions lorsqu'on remettra en route l'alimentation. On dit qu'une **R.O.M.** est **NON VOLATILE**.

- la **R.A.M.** (de **RANDOM ACCESS MEMORY** ou **MEMOIRE A ACCES ALEATOIRE**) contiendra les **DONNEES** (ou **DATA**), sur lesquelles le microprocesseur pourra travailler.

Elle possède la propriété fondamentale de pouvoir être lue (le microprocesseur y trouvera sa matière première pour travailler) et aussi de pouvoir être modifiée (le microprocesseur y stockera ses résultats).

On dit qu'une R.A.M. est à **LECTURE/ECRITURE**.

Notons que lorsque le microprocesseur écrit une donnée en R.A.M., la donnée précédemment présente est remplacée (écrasée et donc perdue) par la nouvelle valeur

Par contre, si on cesse d'alimenter une R.A.M., elle perdra son contenu. Quand on remettra l'alimentation, il y aura absolument n'importe quoi dans la mémoire (contenu à la mise sous tension purement **aléatoire**).

On dit qu'une **R.A.M.** est **VOLATILE**. Cela n'est en fait pas gênant car il existe d'autres supports de mémoire à lecture/écriture comme les disques magnétiques (ou anciennement disquettes), les graveurs RW ou les clefs U.S.B. qui conservent leur contenu même hors alimentation.

Pour retrouver les données, il suffira alors de retransférer une partie du support auxiliaire utilisé vers la R.A.M. :

... voilà un exemple intéressant de travail à donner au microprocesseur ...

Note pour éviter les « incompréhensions » de langage :

Le type de mémoire dont nous parlons ici est appelé couramment **MEMOIRE VIVE** ou **MEMOIRE CENTRALE** (de type R.O.M. ou R.A.M.), réalisée à partir de circuits intégrés à base de semi-conducteur.

Les autres types de mémoire (disques, disquettes, lecteur/graveur, clefs ...) sont appelés **MEMOIRE DE MASSE OU MEMOIRE AUXILLIAIRE**.

On voit ainsi que la mémoire vive est celle qui est le plus proche du microprocesseur, et qui communique directement avec lui.

Une autre notion peut être introduite maintenant. Elle est liée à la façon suivant laquelle les informations sont organisées en mémoire vive. Il faut bien sûr ne pas oublier qu'en électronique numérique, le **QUANTUM D'INFORMATION ELEMENTAIRE** (ou **BIT**) ne peut prendre que deux formes :

- soit le niveau **0** (ou niveau **bas, low, L**) qui correspond souvent à 0V,
(en logique dite positive. En logique négative c'est l'inverse)
- soit le niveau **1** (ou niveau **haut, high, H**) qui correspond à 5V (voire 3,3V),
(en logique dite positive. En logique négative c'est l'inverse)

L'élément électronique de base (hardware) qui permet de réaliser la mémorisation d'un bit élémentaire (software) s'appelle une **BASCULE** (**FLIP FLOP**) dont l'étude relève de la **logique séquentielle**.

Cette notion de bit, nous amène tout naturellement à celle de **REGISTRE** (**REGISTER** en anglais), car en pratique, ces bascules élémentaires sont organisées en registres.

Registres qui seront **intensément** utilisés par la suite.

(registres **à l'intérieur** du microprocesseur, emplacements mémoire qui peuvent être considérés formellement comme des registres **externes** au microprocesseur, registres dans les circuits périphériques programmables...).

Qu'est-ce qu'un registre ? C'est un ensemble de **n** (la plupart du temps $n = 2^k$) bascules regroupées dans une entité qui portera fréquemment un nom (le nom du registre) et qui sera représentée le plus souvent par un schéma comme celui de la figure II.1.

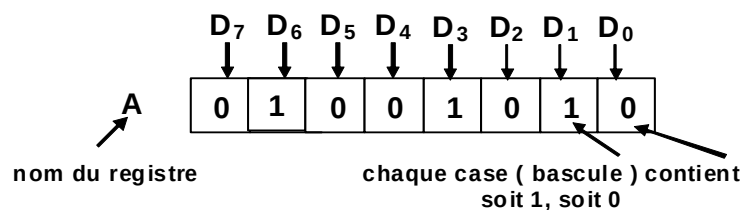


Schéma standard d'un registre pour $n=8$

Figure II.1.

L'entier **n** représente la **taille** du registre et vaudra :

- **n = 4** : on parle de **QUARTET** (ou **NIBBLE** en anglais), cas assez rare,
- **n = 8** : on parle d' **OCTET** (ou **BYTE**), cas très fréquent comme sur le schéma ci-dessus,
- **n = 16** : on parle de **MOT** (ou **WORD**), assez fréquent également,
- **n = 32** : on parle de **DOUBLE MOT** (ou **DOUBLE**)
- voire **une puissance de 2** plus grande sur des architectures plus récentes ou futures.

Comme l'a montré la figure précédente, chaque bit d'un registre est repéré par la lettre D_i , avec $0 \leq i \leq n-1$.

D_0 (le plus à droite) est appelé le **LESS SIGNIFICANT BIT** (**LSB** ou **BIT DE POIDS FAIBLE**).

D_{n-1} (le plus à gauche) est appelé le **MOST SIGNIFICANT BIT** (**MSB** ou **BIT DE POIDS FORT**) : le plus souvent, le MSB sera D_7 (pour un octet) et D_{15} (pour un mot) voire D_{31} (pour un double mot),

Cette séquence $D_{n-1} D_{n-2} \dots D_1 D_0$, formée d'une suite de 0 et de 1, représente en fait le **CODAGE BINAIRE** (en **BASE 2**) de l'information contenue dans le registre.

En fait, c'est le langage **naturel** des microprocesseurs, le seul qu'ils comprennent au final.

Si on veut représenter cette valeur en **DECIMAL** (**BASE 10**), il suffit de calculer :

$$D_{n-1}.2^{n-1} + D_{n-2}.2^{n-2} + \dots + D_2.2^2 + D_1.2^1 + D_0.2^0.$$

On voit donc que l'indice i de chaque D_i représente la puissance de 2 correspondante.

En pratique le codage décimal, très pratique dans la vie de tous les jours, n'est pas adapté dans ce cas et **sera donc très peu** utilisé.

Mais pour éviter les longues séquences de 1 et de 0 du codage binaire, sources d'erreurs et pas très lisibles, on préférera utiliser le **CODAGE HEXADECIMAL** (**BASE 16** et donc avec 16 chiffres différents : de **0** à **9** plus **A, B, C, D, E** et **F**), mieux adapté ici que le système décimal de base 10, car plus proche de l'électronique.

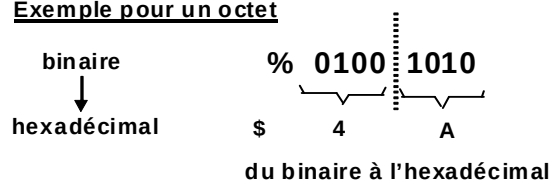
Règle simple à connaître par cœur :

- pour passer du binaire à l'hexadécimal, on regroupe les bits **par paquet de 4**, à partir du **LSB** et on convertit chaque paquet obtenu en utilisant la correspondance entre les 2 dernières colonnes (**binaire** et **hexadécimal**) fournies dans le tableau suivant (la première colonne du codage **décimal** a été rajoutée pour la lisibilité mais ne sert à rien pour appliquer cette règle),
- pour passer de l'hexadécimal au binaire, c'est l'opération inverse...

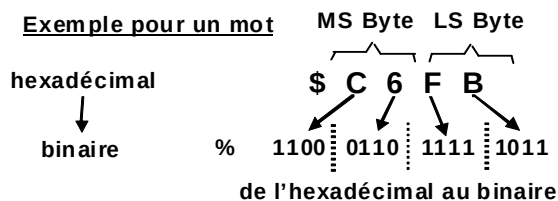
Codage décimal	Codage binaire	Codage hexadécimal
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

Exemple :

Exemple pour un octet



Exemple pour un mot



Je vois trop souvent des étudiants qui ne savent pas appliquer cette règle, pourtant simple, et sont du coup obligés de passer par (l'inutile) étape intermédiaire du décimal et de faire de longues suites de divisions successives par 2 pour aboutir finalement au binaire : perte de temps, manque d'efficacité et risque d'erreurs fréquentes ...

Passer **donc un peu de temps** à faire quelques conversions pour vous familiariser avec cette méthode.

**Il est important de bien maîtriser ce codage, proche du hardware et de l'électronique
et de passer facilement du binaire à l'hexadécimal et réciproquement.
Vous vous en rendrez rapidement compte en TP !!!**

Remarque et convention :

Sur l'exemple précédent :

- le symbole % devant les valeurs 01001010 ou 1100011011111011 indique qu'il s'agit d'un codage binaire,
- le symbole \$ devant les valeurs 4A ou C6FB indique qu'il s'agit d'un codage hexadécimal (en langage C, ce serait **0x4A** ou **0xC6FB**) , pour éviter la confusion avec le décimal (si les digits sont des chiffres et pas des lettres),
- **l'absence de symbole** signifiant que la valeur doit s'interpréter en décimal.

Par exemple :

10 (sans symbole) = $(10)_{10}$ (décimal) = \$0A = % 1010 $\neq$ \$10 ($= (16)_{10}$) $\neq$ %10 ($= (2)_{10}$)

En pratique, on peut aussi utiliser **l'hexadécimal pour nommer** les différents bits d'un octet :

D₀ s'appelle le bit **\$01** (correspond au masque % 0000 000**1** : seul D₀ à 1, les autres à 0).

D₁ s'appelle le bit **\$02** (correspond au masque % 0000 00**10** : seul D₁ à 1, les autres à 0).

D₂ s'appelle le bit **\$04** (correspond au masque % 0000 0**100** : seul D₂ à 1, les autres à 0).

D₃ s'appelle le bit **\$08** (correspond au masque % 0000 **1000** : seul D₃ à 1, les autres à 0).

D₄ s'appelle le bit **\$10** (correspond au masque % 000**1** 0000 : seul D₄ à 1, les autres à 0).

D₅ s'appelle le bit **\$20** (correspond au masque % 00**10** 0000 : seul D₅ à 1, les autres à 0).

D₆ s'appelle le bit **\$40** (correspond au masque % 0**100** 0000 : seul D₆ à 1, les autres à 0).

D₇ s'appelle le bit **\$80** (correspond au masque % **1000** 0000 : seul D₇ à 1, les autres à 0).

L'autre façon de nommer les différents bits d'un octet est de donner **l'indice i** du **bit D_i**.

Par exemple **D₀** est le **bit 0**, **D₁** est le **bit 1**,, **D₇** est le **bit 7**.

Mais le repérage précédent d'un bit avec des masques est à mon avis plus parlant et plus évocateur car il fait le lien avec l'hexadécimal et permet, grâce au masque, de repérer le bit à 1 positionné dans l'octet. Il serait utilisé préférentiellement pour définir des masques.

La plupart des registres possèdent deux propriétés essentielles :

- on peut les **lire** (accès en **lecture**) : pour accéder à leur contenu (sans le modifier),
- on peut y **écrire** (accès en **écriture**) : pour modifier leur contenu. Notons que dans ce cas la suite de 0 et 1 précédemment présente est remplacée (écrasée et donc perdue) par la nouvelle séquence qui vient d'être écrite.

De façon générale, on accède au contenu des registres en mode dit « **parallèle** », c'est-à-dire que tous les bits D_i sont lus (ou écrits) **tous en même temps**, en un seul coup d'horloge, comme indiqué sur la figure II.2. :

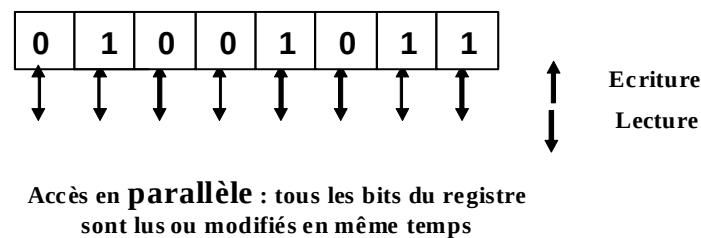


Figure II.2.

Ce mode d'accès, nécessitant au moins 8 fils (pour un octet) plus un fil pour la masse est le plus rapide mais sera en général réservé lorsque les composants sont positionnés sur la même carte électronique où le nombre de fils (en fait des pistes de cuivre sur le circuit imprimé) n'est pas un problème en soi.

Par opposition à un accès dit en « **série** » où les bits D_i sont lus (ou écrits) **les uns après les autres**, ce qui est donc nécessairement plus lent, car nécessite 8 coups d'horloge, toujours pour un octet. (cf. figure II.3.). On parle alors de **registre « à décalage »** (**shift register**).

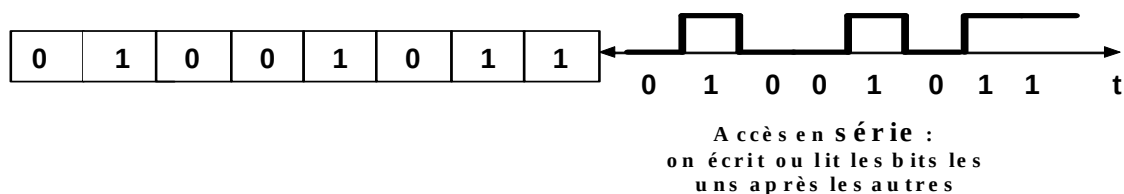


Figure II.3.

Par contre, l'accès série peut se faire avec un minimum de deux fils : un fil pour l'entrée/sortie des bits plus un fil pour la masse.

En conclusion, l'accès en parallèle, **plus rapide**, est donc celui qui sera utilisé le plus fréquemment, surtout au niveau de l'architecture électronique et des échanges à **l'intérieur** des systèmes. (l'USB qui est en mode série, sert dans les communications avec **l'extérieur**).

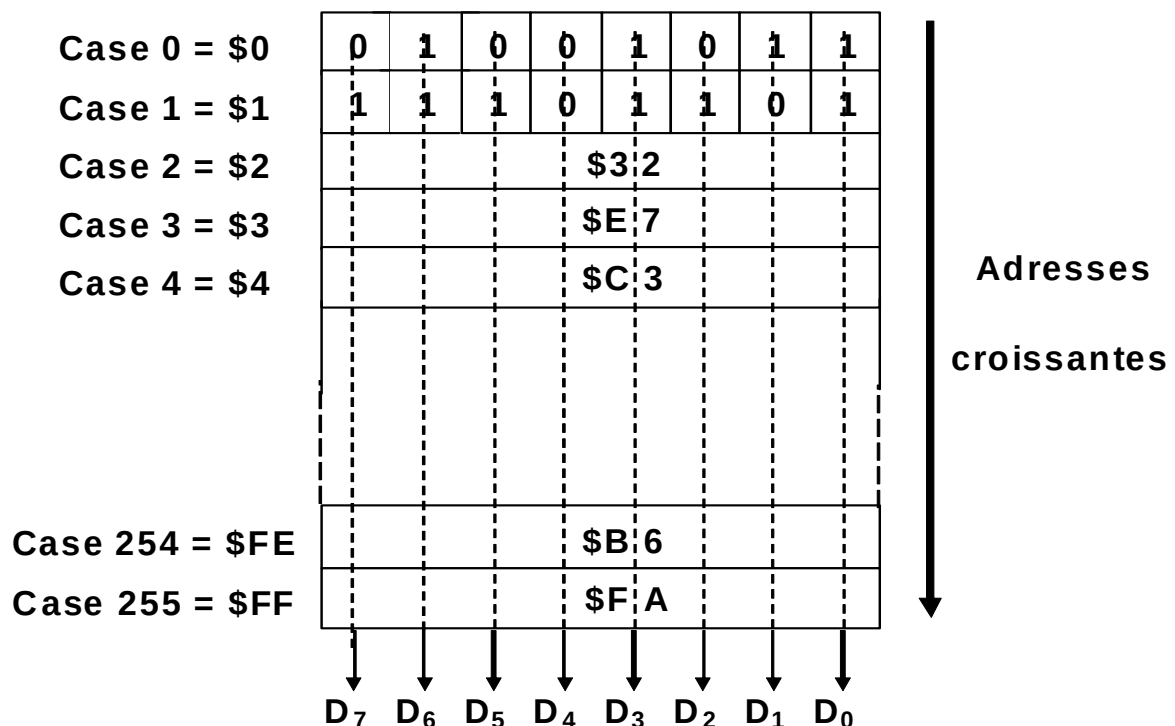
Certains registres ont aussi la propriété supplémentaire de pouvoir être utilisés en mode **compteur**, c'est-dire qu'il existe la possibilité d'augmenter de 1 leur contenu (**incréméntation** ou **count up**) ou de diminuer de 1 leur contenu (**décréméntation** ou **count down**).

Il est intéressant maintenant de faire le lien entre registres de base et mémoire vive.

Retour sur les mémoires vives :

Il faut se représenter mentalement une mémoire **vive** comme une gigantesque " armoire " munie de tiroirs (il peut y en avoir jusqu' à plusieurs millions), chaque tiroir étant vu comme un registre c'est-à-dire lui-même divisé en **n** compartiments élémentaires, chaque compartiment pouvant, comme on l'a vu plus haut stocker un **1** ou un **0**. En pratique, on aura souvent **n = 8**, comme sur la figure II.4.

Et pour représenter cette image de la mémoire « armoire », on rencontrera souvent dans la littérature technique, le schéma assez **intuitif et parlant** fourni figure II.4.



**Représentation "classique" d'une zone
de mémoire vive
(ici de taille (très modeste) de 256 octets)**

Figure II.4.

Avec cette image en tête :

- l'armoire complète représente le circuit intégré mémoire vive,
- chaque tiroir est un **EMPLACEMENT MEMOIRE** (on dit aussi **CASE MEMOIRE**),
- chaque compartiment contient un bit (**1** ou **0**).

Pour pouvoir être différenciée de ses voisines, chaque case mémoire est caractérisée par un numéro qui lui est propre et que l'on appelle son **ADRESSE** (un peu comme chaque individu possède sa propre adresse postale ou son adresse mail).

Dans un même système, deux cases différentes ne pourront pas avoir la même adresse (ce qui entraînerait un conflit de fonctionnement, comme si deux personnes différentes avaient la même adresse mail ...)

Par contre, une case mémoire peut avoir plusieurs adresses (une parmi celles-ci pourra être utilisée de façon courante, les autres sont alors appelées **ADRESSES FANTÔMES** (ou **GHOST**)), un peu comme quand vous possédez plusieurs adresses mail.

De plus, certaines adresses peuvent ne correspondre à aucune case mémoire physique réelle. (une adresse mail choisie « au hasard » ne correspondant à aucune personne physique : à ne pas utiliser).

Ce schéma appelle quelques commentaires supplémentaires.

Comme on l'a dit, chaque case mémoire a bien été repérée par son numéro (son **adresse**), inscrit sur la gauche de la figure. On voit par exemple que la case **d'adresse 0** contient la valeur binaire constituée ici d'une séquence de 8 bits dont la valeur vaut **% 0 1 0 0 1 0 1 1**, valeur choisie ici totalement au hasard.

Le contenu des cases 2, 3, 4 ... a été mis en hexadécimal sur la figure précédente ainsi que les adresses des différentes cases mémoire ... car en pratique, les adresses des différentes cases mémoire, ainsi que leur contenu sont le plus souvent exprimés en hexadécimal. On commence déjà à sentir l'importance et l'utilisation fréquente du codage hexadécimal.

Mais attention :

Il ne faut pas confondre (source d'erreurs fréquentes): le <u>nombre « inscrit à côté de la case »</u> (son adresse, l'adresse de la donnée) avec <u>le nombre « à l'intérieur de la case »</u> (son contenu : la valeur de la donnée).
--

Comme sur le schéma de la figure II.4. précédente, on placera souvent les **adresses basses** (les plus faibles) **en haut**, et les **adresses hautes** (les plus fortes) **en bas**, comme le rappelle la flèche « **adresses croissantes** » sur la droite de cette figure.

De plus, comme le montre le schéma de la figure II.4., les bits correspondant au contenu des cases mémoire sont souvent repérés par la séquence **D₇ D₆ D₅ D₄ D₃ D₂ D₁ D₀** .

Comme les registres, les mémoires possèdent la propriété de fournir leur contenu de chacune de leurs cases (via les lignes **D_i** décrites précédemment) si on leur indique l'adresse de cette case.

Pour cela, elles possèdent un certain nombre de **broches d'entrées** (nommées **A_i**) sur lesquelles la CPU devra positionner le numéro de la case mémoire à laquelle elle veut accéder.

Pour comprendre le principe de la structure et du fonctionnement d'une mémoire vive, prenons un exemple simple :

supposons qu'il existe 4 **entrées** nommées **A₃ A₂ A₁ A₀** .
(**A₀** représentant toujours le **LSB** du bus d'adresses).

Chacune de ces entrées pouvant valoir :

- soit **0** (par exemple 0 V),

- soit **1** (par exemple 5 V),

il y aura donc $2^4 = 16$ combinaisons différentes pour les **A_i**, et donc **16 cases mémoire différentes** dans ce composant.

Si on veut lire par exemple dans la case d'adresse 7 il suffira pour la CPU d'imposer la combinaison **A₃ A₂ A₁ A₀ = 0 1 1 1**.

(à l'intérieur de la mémoire, il existe en fait un décodeur qui permet de sélectionner **uniquement** la case dont le numéro correspond à la valeur présente en binaire sur les entrées **A_i**).

On voit donc que dans le cas général où on a des entrées **A_r, A_{r-1} ... A₁ A₀** c'est à dire **r+1** entrées d'adresses de case mémoire, la **CAPACITE TOTALE** (ou **taille**, et donc le nombre total de cases) de cette mémoire sera de 2^{r+1} cases mémoire. (une puissance de 2).

Sur le vocabulaire courant, concernant les tailles mémoire :

Puissance de 2 (ou de ≈ 10)	Capacité exacte	Arrondi à
$2^{10} \approx 10^3$	1 024	1 K(ilo)
$2^{20} \approx 10^6$	1 048 576	1 M(éga)
$2^{30} \approx 10^9$	1 073 741 824	1 G(iga)
$2^{40} \approx 10^{12}$	$1,099511627 \cdot 10^{12}$	1 T(éra)

On définit aussi la notion de **TEMPS D'ACCES** : durée au bout de laquelle la case mémoire fournit son contenu à l'extérieur du composant. Notons que ce temps d'accès est plus élevé pour les ROM que pour les RAM et ceci pour des raisons technologiques non détaillées dans cette présentation succincte.

En pratique, on a vu qu'il existe 2 grandes familles de mémoires vives. Précisons les ici :

- les mémoires vives de type **RAM** : **R**andom **A**ccess **M**emory :

- elles sont en **lecture et en écriture**. (**READ and WRITE**)

Notons que lorsque la CPU écrit une donnée en R.A.M., la donnée précédemment présente est remplacée (écrasée et donc perdue) par la nouvelle valeur.

- elles perdent leur contenu lorsque leur alimentation est coupée : on dit qu'elles sont **volatiles**. A chaque mise sous tension, elles contiennent n'importe quoi !!!

- elles servent en général à stocker des données (mais aussi des programmes en cours de développement),

- elles peuvent être très rapides, surtout pour les RAM statiques.

- les mémoires vives de type **ROM** : **R**ead **O**nly **M**emory.

- elles ne sont **qu'en lecture**

- elles conservent leur contenu même lorsque leur alimentation est coupée : on dit qu'elles sont **non volatiles**. A chaque mise sous tension, elles contiennent toujours la même chose : les valeurs qui y ont été introduites quand on les a programmées !!!

- elles servent à stocker les programmes systèmes qui s'exécuteront à chaque mise sous tension (et parfois même certaines données importantes à conserver après chaque coupure d'alimentation : mot de passe, paramètres de configuration ...)

- elles sont, en général, plus lentes que les RAM.

Remarque sur le vocabulaire :

Le terme R.A.M. présenté de cette façon par opposition à celui de R.O.M. peut prêter parfois à confusion : il aurait été mieux de les appeler R.W.M. (Read and Write Memory), car une R.O.M. est aussi à " Random Access ". En effet, la notion de " Random Access " (Acces Aléatoire) rappelle que toutes les cases de ce type de mémoire sont accessibles directement par leur adresse (par opposition aux mémoires séquentielles où l'accès est effectué par approches successives).

La dernière chose à signaler dans cette présentation simplifiée des mémoires, est l'existence d'une autre **entrée** nécessaire notamment pour les RAM. qui peuvent être utilisées soit en écriture, soit en lecture.

Cette autre entrée sera souvent (mais pas toujours) appelée $R/\overline{W}$ (comme READ / $\overline{WRITE}$), notation signifiant que si :

- $R/\overline{W} = 1$: phase de **lecture** (**READ**) d'une case (sélectionnée par les A_i) de la RAM,
- $R/\overline{W} = 0$: phase de **d'écriture** (**WRITE**) dans une case (sélectionnée par les A_i) de la RAM.

Dans le cas des ROM, qui ne fonctionnent pas en écriture, une entrée de type $\overline{OE}$ (comme **OUTPUT ENABLE**) est présente à la place de la commande $R/\overline{W}$ des RAM.

Cette entrée obéit à la logique suivante :

- $\overline{OE} = 1$, le contenu de la case mémoire sélectionnée n'est pas présente sur les D_i ,
- $\overline{OE} = 0$, le contenu de la case mémoire sélectionnée est présente sur les D_i , donc pendant une phase de lecture de la ROM.

En résumé, le tableau ci-dessous rappelle les principales propriétés des mémoires vives :

Propriétés	RAM	ROM
Surtout pour :	Les données	Le programme (instructions)
Mode d'accès	Aléatoire	Aléatoire
Type d'accès	Lecture et écriture	Lecture
Coupage d'alimentation	Volatile	Non volatile
Vitesse (cf. temps d'accès)	Rapides (surtout RAM statiques)	Plus lentes que les RAM
Capacité pour $A_r, A_{r-1} \dots A_1 A_0$	2^{r+1} cases mémoire	2^{r+1} cases mémoire

Question récapitulative :

Qu'avons nous défini aux termes de cette approche " intuitive " ?

Réponse :

La mémoire vive sera un circuit intégré qui possèdera :

- un certain nombre de broches (ou pattes, pins) d'adresses : $A_0 A_1 \dots A_r$ (r est un entier dépendant de la capacité totale de la mémoire). Ces broches sont des **entrées** du circuit intégré mémoire sur lesquelles il faudra placer le numéro de la case avec laquelle on souhaite travailler,
- une broche au moins en **entrée** pour différencier la lecture de l'écriture dans le cas d'une RAM (broche $R/\overline{W}$), ou pour valider les D_i en sortie dans le cas d'une ROM (broche $\overline{OE}$),
- un certain nombre de broches de données : $D_0 D_1 \dots D_t$ (t est un entier dépendant de la taille de chaque case mémoire, valant souvent 8, 16 ou 32). Ces broches sont utilisées :
 - comme des **sorties** pour la **R.O.M.** : elles reflètent alors le contenu en binaire de la case de mémoire adressée par les A_i ,
 - comme des **entrées / sorties** pour la **R.A.M.** :
 - en **sortie** : idem R.O.M.
 - en **entrée** : on placera sur ces broches la combinaison que l'on veut écrire dans la case mémoire adressée par les A_i .

Ce qui nous conduit au synoptique de brochage minimal de la figure II.5. suivante où l'on a également rajouté une broche pour le **VCC** (pour l'alimentation du circuit intégré) et le **GND** (pour la masse).

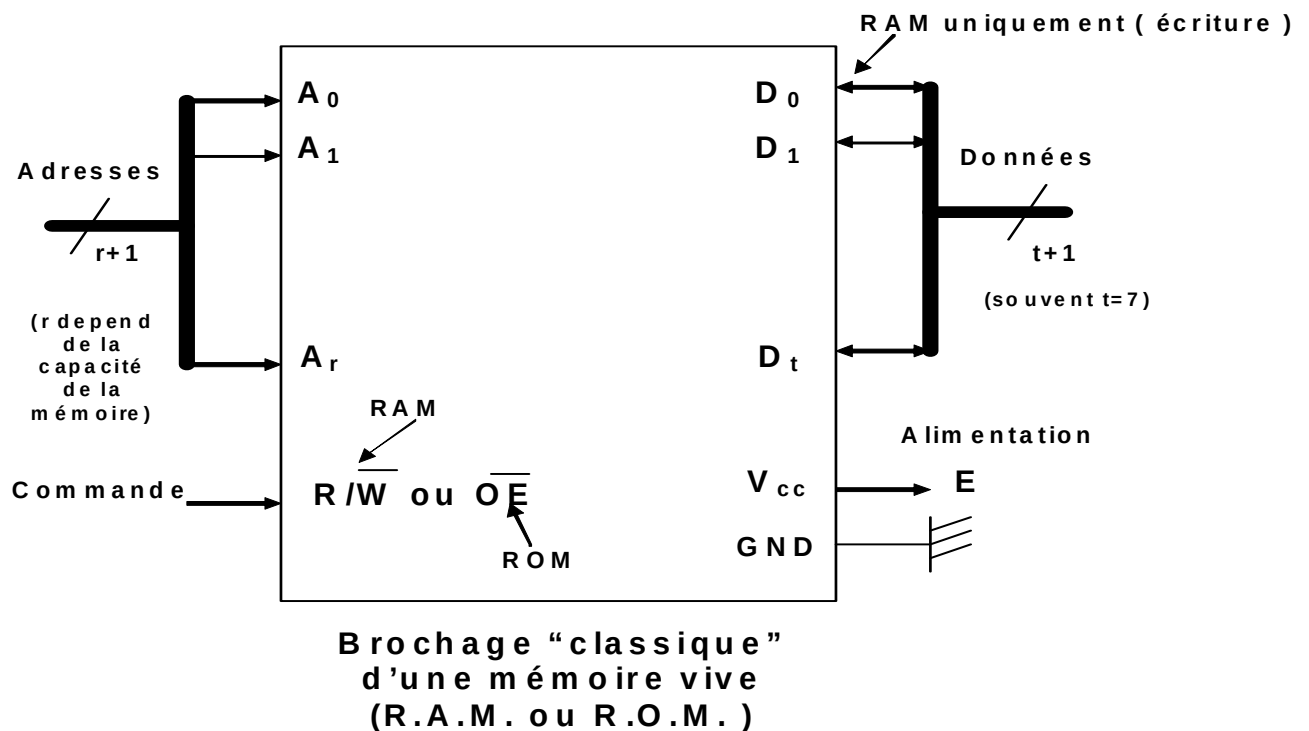


Figure II.5.

La plupart des mémoires réelles possèdent quasiment ce type de brochage. A titre indicatif, voici un extrait d'une documentation constructeur pour une ROM. (de type EPROM).

2732 32K (4K x 8) UV ERASABLE PROM

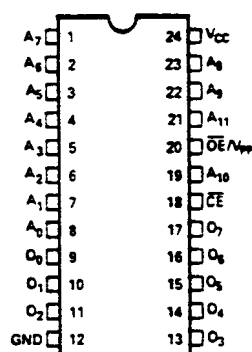
- Single +5V $\pm$ 5% Power Supply
- Output Enable for MCS-85™ and MCS-86™ Compatibility
- Fast Access Time: 450ns Max.
- Low Power Dissipation:
150mA Max. Active Current
30mA Max. Standby Current
- Pin Compatible to Intel® 2716 EPROM and 2332/2364 ROMs
- Completely Static
- Simple Programming Requirements
— Single Location Programming
— Programs with One 50ms Pulse
- Three-State Output for Direct Bus Interface

The Intel® 2732 is a 32,768-bit ultraviolet erasable and electrically programmable read-only memory (EPROM). The 2732 operates from a single 5-volt power supply, has a standby mode, and features an output enable control. For production, the pin compatible 2332 and 2364 ROMs are available. The total programming time for all bits is three and a half minutes. All these features make designing with the 2732 in microcomputer systems faster, easier, and more economical.

An important 2732 feature is the separate output control, Output Enable ($\overline{OE}$), from the Chip Enable control ($\overline{CE}$). The $\overline{OE}$ control eliminates bus contention in multiple bus microprocessor systems. Intel's Application Note AP-30 describes the microprocessor system implementation of the $\overline{OE}$ and $\overline{CE}$ controls on Intel's 2716 and 2732 EPROMs. AP-30 is available from Intel's Literature Department.

The 2732 has a standby mode which reduces the power dissipation without increasing access time. The maximum active current is 150mA, while the maximum standby current is only 30mA, an 80% savings. The standby mode is achieved by applying a TTL-high signal to the $\overline{CE}$ input.

PIN CONFIGURATION



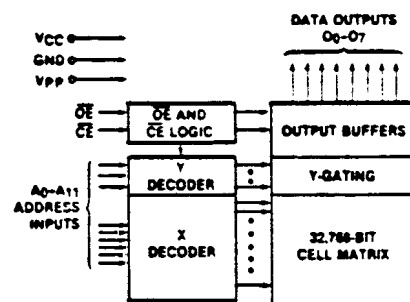
PIN NAMES

A ₀ -A ₁₁	ADDRESSES
$\overline{CE}$	CHIP ENABLE
$\overline{OE}$	OUTPUT ENABLE
O ₀ -O ₇	OUTPUTS

MODE SELECTION

MODE	PINS	$\overline{CE}$ (18)	$\overline{OE}/V_{pp}$ (20)	V _{CC} (24)	OUTPUTS (9-11,13-17)
Read		V _{IL}	V _{IL}	+5	D _{OUT}
Standby		V _{IH}	Don't Care	+5	High Z
Program		V _{IL}	V _{pp}	+5	D _{IN}
Program Verify		V _{IL}	V _{IL}	+5	D _{OUT}
Program Inhibit		V _{IH}	V _{pp}	+5	High Z

BLOCK DIAGRAM



Exercice :

- retrouver les différentes broches vues dans les généralités (voir le **PIN CONFIGURATION**)
- quelle est la taille d'une case mémoire ? Justifier.
- quelle est la capacité mémoire de ce composant ? Justifier. Faire le lien avec le **32** de la référence 27**32**.
- retrouver le rôle de l'entrée $\overline{OE}$ (voir le tableau **MODE SELECTION**)

Noter l'existence à l'intérieur du composant (voir le **BLOCK DIAGRAM**) du décodeur.

Rappeler quel est son rôle ?

Retour sur le brochage « classique » d'une mémoire de la figure II.5. :

En fait, "il manque encore " un petit quelque chose ", une patte supplémentaire importante pour assurer un fonctionnement correct.

Question :

Quelle broche peut-il bien manquer ?

Réponse :

La réponse sera fournie plus loin, quand on abordera le principe du décodage d'adresses.

II.1.3. L'EXTERIEUR (OU ENVIRONNEMENT)

Il manque maintenant un dernier élément à notre système constitué de son microprocesseur et de sa (ses) mémoire(s).

Toute carte électronique est utilisée dans un but précis, le plus souvent pour agir sur son environnement en fonction de celui-ci.

Cette fonctionnalité est le plus souvent regroupée sous le terme " **interface ou communication avec l'extérieur** " ou encore " **ENTREES/SORTIES** ". (ou E/S ou **INPUT/OUTPUT** ou **I/O**).

On pourrait penser que le microprocesseur peut également et directement s'occuper aussi de ce travail de gestion de la communication avec l'extérieur.

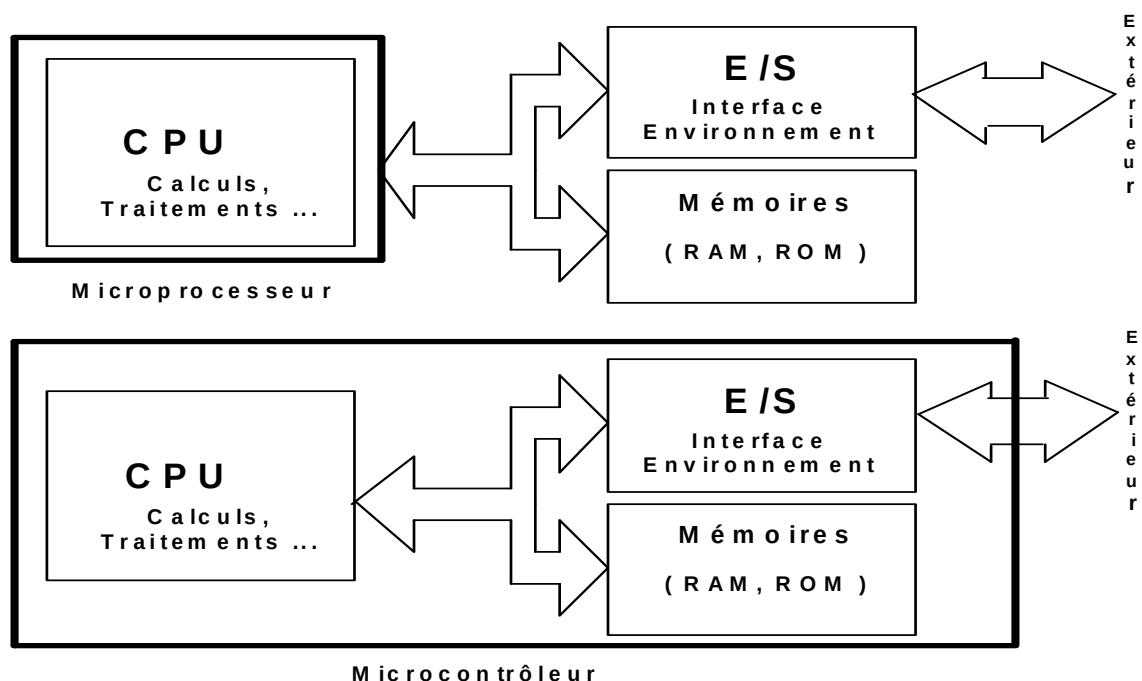
C'est ce qui se passe avec les **MICROCONTRÔLEURS** mais pas avec les **MICROPROCESSEURS**.

En plus, à l'intérieur d'un microcontrôleur, on trouvera aussi de la mémoire (RAM et ROM).

Ce qui au final permet, dans un seul boîtier, d'intégrer une architecture complète et donc de simplifier la conception hardware, de réduire les coûts et de diminuer la taille du système. (cf. figure II.6.).

Ceci sera intéressant pour les applications relativement simples.

Car **pour une surface donnée** de silicium, un microprocesseur aura toujours une CPU plus complète et plus performante que celle d'un microcontrôleur (à cause de la place prise sur cette surface par le bloc E/S réduisant la taille et donc quelque part les performances de la CPU du microcontrôleur par rapport à celle d'un microprocesseur).



CPU : Central Process Unit :là où sont faits les divers traitements
C'est le « cœur » des micro /processeurs/contrôleurs

Figure II.6.

Avec un microprocesseur l'idée principale est de le décharger de ce travail d'interface avec l'environnement pour qu'il puisse faire autre chose en parallèle.

Il existe ainsi sur le marché un grand nombre de circuits intégrés, que l'on appelle circuits **PERIPHERIQUES** et qui ont été conçus pour réaliser la plupart des fonctions de communication et d'interface avec l'environnement. En voici quelques exemples (liste non exhaustive) :

- gérer une liaison avec un clavier, un écran, une imprimante,
- gérer une liaison entre deux ou plusieurs systèmes informatiques,
- lire l'état de capteurs tout ou rien (boutons poussoirs, interrupteur de fin de course ...),
- acquérir une grandeur analogique (mesure d'une température, d'une tension ...),
- commander des actionneurs tout ou rien (relais, afficheurs optiques ...)
- fabriquer une tension analogique : consigne d'un variateur de vitesse pour un moteur électrique,
- mesurer un intervalle de temps ou compter des événements extérieurs : timers ...

Remarque sur le vocabulaire utilisé ici :

Le terme périphérique est aussi rencontré dans la littérature informatique pour désigner du " gros " matériel : l'imprimante, le clavier ou l'écran peuvent être appelés des périphériques du micro-ordinateur. Au niveau de notre présentation, nous appelons ici périphérique, le circuit intégré qui permettra de piloter ce " gros " matériel.

Vu la variété des fonctions réalisées par ces circuits périphériques, on se doute bien qu'ils seront tous spécifiques et que chaque cas sera un petit peu un cas particulier. Nous allons pourtant ici essayer de présenter la « philosophie » générale de ce genre de composant en essayant d'extraire les éléments fondamentaux que l'on retrouvera à peu près dans tous les circuits de cette famille.

On peut par exemple supposer, sans trop prendre de risques, que ces circuits, qui effectuent un certain travail sont dotés d'une certaine " autonomie " leur permettant de pouvoir :

a- s'adapter à des normes ou de configurations différentes (par exemple un contrôleur de liaison série doit pouvoir travailler à différentes vitesses de communication),

b - recevoir des ordres émanant du microprocesseur, qui reste quand même toujours le "vrai patron", (par exemple pour initialiser un circuit ou pour lui communiquer une tâche à accomplir),

c - communiquer le résultat de leur travail au microprocesseur (par exemple dans le cas d'un timer signaler qu'un intervalle de temps donné s'est écoulé).

Ces trois fonctionnalités, en général nécessaires, impliqueront l'existence de **registres internes** dans tout circuit périphérique. Leur nombre pourra varier, mais on pourra toujours les classer dans les trois grandes familles ci-dessous, correspondant aux trois fonctionnalités précédentes :

a - REGISTRE DE MODE (MODE REGISTER) :

En fonction de son contenu (défini par le microprocesseur, et donc par le programme, pendant une phase appelée **INITIALISATION** du composant périphérique. ou plus succinctement " **PHASE D'INIT** "), le circuit périphérique pourra s'adapter à telle ou telle configuration matérielle ou logicielle ou telle ou telle norme existante.

L'intérêt est évident, puisqu'ainsi le même composant pourra être utilisé pour plusieurs normes ou standards d'applications. Sinon, il aurait fallu changer de composant pour la moindre modification.

b - REGISTRE DE CONTRÔLE (CONTROL REGISTER) :

De façon très générale pour recevoir les ordres émanant du microprocesseur ou pour faciliter la surveillance du composant par le microprocesseur.

c - REGISTRE D'ETAT (STATUS REGISTER) :

Son contenu reflète, à un instant donné, l'état d'avancement du travail du circuit périphérique, ou le résultat de telle ou telle tâche élémentaire ou intermédiaire réalisée par ce circuit périphérique. Par exemple, si un caractère arrive sur une liaison, un bit particulier de ce registre d'état pourra passer à 1 pour signaler au microprocesseur qu'un nouveau caractère vient d'arriver et qu'il faut donc aller le lire, car sinon tout caractère suivant arrivant juste après risquerait de l'écraser.

A cause de l'existence de ces registres internes, les circuits périphériques sont des composants **PROGRAMMABLES**. Chacun de ces registres internes, comme pour les cases de mémoire, sera caractérisé par un numéro (ou adresse) qui lui sera propre. Ces registres doivent pouvoir être lus (CONTROL et STATUS REGISTER) et/ou modifiés à loisir par le microprocesseur (MODE et CONTROL REGISTER). Ainsi, ils seront de la même technologie que celle utilisée pour la R.A.M.

Malheureusement dans la pratique, les choses ne sont pas toujours aussi simples que dans ces généralités :

- on peut trouver plusieurs registres pour chacune de ces trois fonctionnalités de base,
- un même registre pourra servir par exemple pour le mode (certains bits de ce registre) et le contrôle (les autres bits de ce même registre) ou d'autres types de combinaisons, et ceci dans un souci d'optimiser la quantité d'informations programmables pour un nombre de registres donné,
- il peut y avoir d'autres registres liés au type de travail que le composant effectue et aux spécificités de chaque circuit périphérique,
- à notre époque de " forte intégration ", il y aura souvent plusieurs fonctionnalités regroupées dans le même circuit périphérique ...

Le domaine des circuits périphériques est à mon avis un des plus délicats de l'informatique et nécessite souvent de consulter et d'exploiter beaucoup de documentations constructeurs pour comprendre les spécificités de fonctionnement de chaque circuit périphérique.

Grâce à cette présentation rapide et très générale, nous pouvons maintenant donner un synoptique minimal du brochage d'un circuit périphérique, constitué par :

- le **GND** et le **VCC**, on n'y coupe pas,
- un certain nombre de broches d'adresses : $A_0 A_1 \dots A_p$ (p est un entier dépendant du nombre de registres internes du circuit, ici au maximum 2^{p+1}). Ces broches sont des **entrées** du circuit sur lesquelles il faudra placer le numéro du registre interne avec lequel on souhaite travailler,
- une broche au moins en **entrée** pour différencier la lecture de l'écriture dans le registre interne adressé par les A_i : broche $R/\overline{W}$,
- un certain nombre de broches de données : $D_0 D_1 \dots D_{t'}$ (t' est un entier dépendant de la taille de chaque registre interne, valant souvent 8). Ces broches sont utilisées en **entrées** / **sorties** comme pour la R.A.M. :
 - en **entrée** : on placera sur ces broches la combinaison que l'on veut écrire dans le registre interne adressé par les A_i .
 - en **sortie** : on récupèrera sur ces broches le contenu du registre adressé par les A_i .
- éventuellement une entrée **RESET** (le reset est fait au niveau haut de cette entrée) ou $\overline{\text{RESET}}$ (le reset est fait au niveau bas de cette entrée) qui permet, à la mise sous tension (ou par programme) d'initialiser tous les registres internes du composant avec des valeurs " garanties " par le constructeur du circuit et ainsi d'éviter des fonctionnements aléatoires au démarrage. Ainsi tous les composants périphériques d'une même référence et tous les " programmeurs du monde " partiront du même état initial, condition minimum pour une certaine compatibilité,
- d'autres broches **d'entrée (E)**, de **sortie (S)** et/ou d'**entrée/sortie (E/S)**, dépendant du type de fonctionnalités réalisées par le circuit périphérique et lui permettant de communiquer avec son environnement extérieur.

Ce qui nous conduit au synoptique de brochage de la figure II.7. ci-dessous :

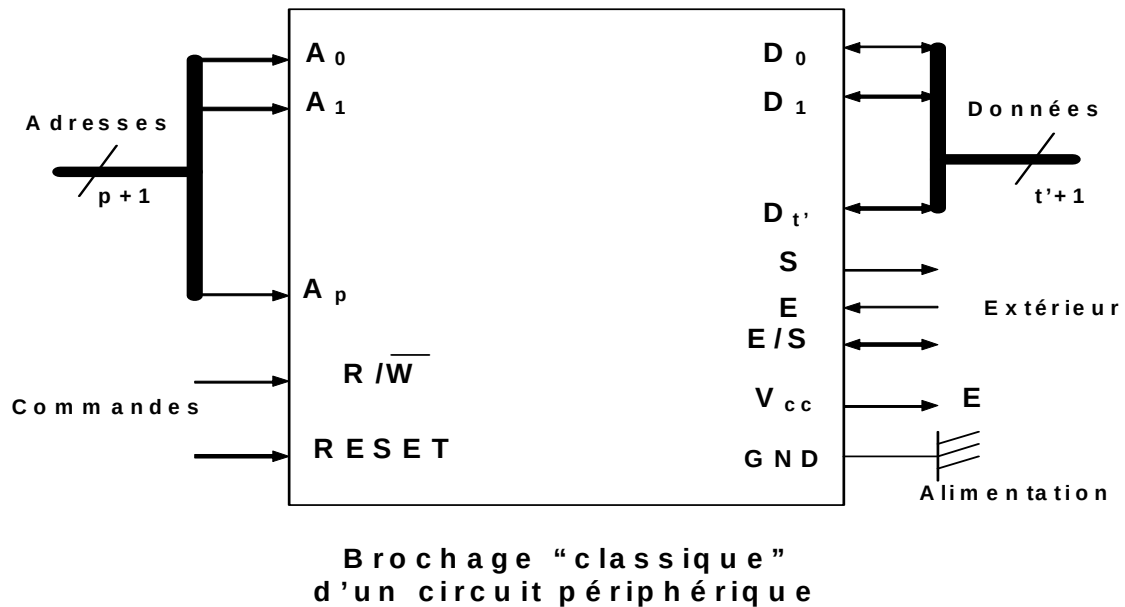


Figure II.7.

On verra que, là aussi, il manque encore " un petit quelque chose ".

Question :

Quelle broche peut-il bien manquer ?

Comme précédemment, la réponse sera fournie plus loin, dans la partie sur le décodage d'adresses...

Au stade actuel, nous pouvons maintenant apporter d'autres éléments pour enrichir la partie **II.1.1.** sur le microprocesseur.

Les notions de broches pour les adresses et pour les données introduites pour les mémoires et les circuits périphériques doivent pouvoir être généralisées au niveau du microprocesseur.

En effet, n'oublions pas que le microprocesseur est l'exécutant de notre système. Il possède en outre aussi l'importante propriété suivante :

Il sait, de façon quasi-automatique, rechercher et trouver tout seul en mémoire le travail qu'on lui demande d'exécuter.

On verra comment plus loin quand on donnera des éléments concernant la structure interne d'un microprocesseur, et notamment quand on introduira l'important et fondamental registre **POINTEUR de PROGRAMME** (ou **COMPTEUR ORDINAL** ou **PROGRAM COUNTER** ou **INSTRUCTION POINTER**, tous ces termes étant des synonymes)

Comment s'y prend-il ? En fait, il possède :

- un certain nombre de broches d'adresses : $A_0 A_1 \dots A_m$ (m est un entier dépendant de la **CAPACITE D'ADRESSAGE** du microprocesseur, c'est-à-dire du nombre maximum de cases mémoire différentes auxquelles il peut accéder). Ces broches sont des **sorties** du microprocesseur sur lesquelles il placera le numéro de la case avec laquelle il souhaite communiquer,

- un certain nombre de broches de données : $D_0 D_1 \dots D_{m'}$ (m' est un entier qui permet de caractériser le type (la génération) du microprocesseur. Il vaudra 7 (soit 8 broches pour les données) pour les microprocesseurs dits **8 bits**, 15 (soit 16 broches) pour les microprocesseurs dits **16 bits** etc ...). Ces broches sont des **entrées / sorties** :

- en **entrée** : il lira sur ces broches le contenu de la case adressée,
- en **sortie** : il placera sur ces broches la combinaison à écrire dans la case adressée,

- une broche au moins en **sortie** pour différencier la phase de lecture de celle d'écriture en mémoire ou dans un composant périphérique : broche $R/\overline{W}$,

- éventuellement une entrée **RESET** (ou $\overline{RESET}$) qui permettra d'initialiser le microprocesseur (elle permet entre autre de placer sur ses **sorties** A_i l'adresse de la toute première instruction à exécuter, qu'il faudra bien sûr placer en ROM. Pourquoi ?),

- et bien sûr le **GND** et le **VCC** comme toujours ...

Ce qui nous conduit au synoptique de brochage de la figure II.8. ci-dessous :

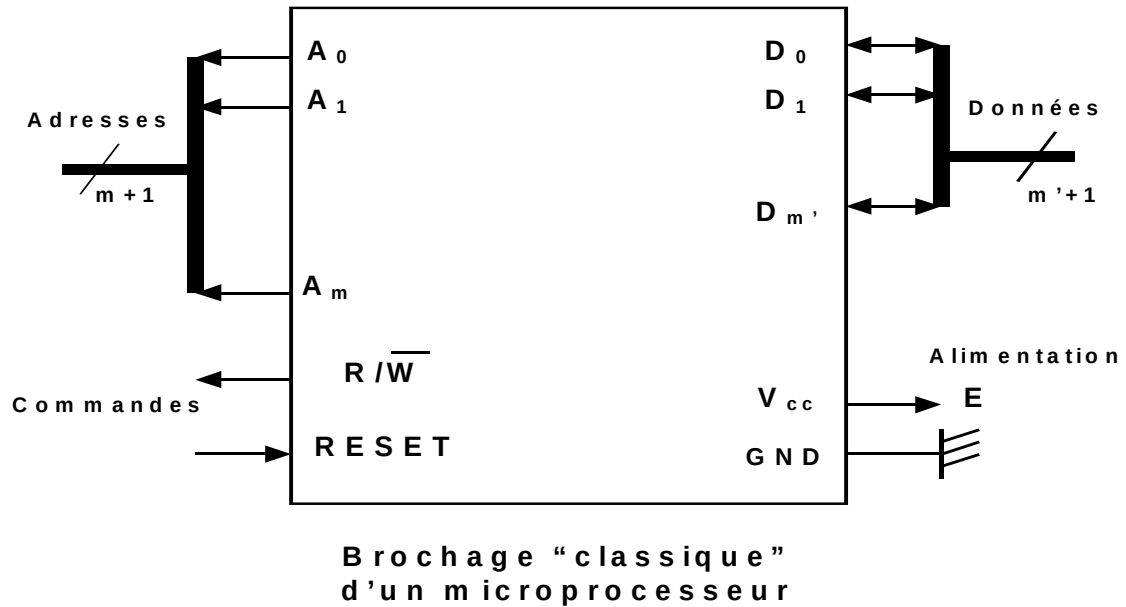


Figure II.8.

On verra qu'il manque encore " un petit quelque chose ".

Exercice :

Quelle broche peut-il bien manquer ?

Aide : n'oubliez pas qu'on travaille en logique séquentielle ...

Avant de passer au chapitre suivant, il me semble intéressant de faire un petit historique concernant l'évolution des microprocesseurs et de leurs principales performances.

Périodes	Années 80		Années 90		Années 2000				
	80	85	85	90	95	2000	2000	10	
Microprocesseurs de référence	Z80 Zilog 8080 Intel		68 000 Moto 8086 Intel		68 030 Moto 386, 486 Intel		Power PC Pentium I		Série Pentium
Fréquences	1 MHz		20 MHz		400 MHz		1 GHz		4 GHz
Type	8 bits		16 bits		32 bits		64 bits		128 bits
Capacité d'adressage	64 Ko		1 Mo		32 Mo		1 Go		1 To
Performances sur une base 1 en 1980 (ordre de grandeur)	1		15		250		4000		250 000

Rappeler vous la loi (ordre de grandeur) de Moore :

Les performances en électronique/informatique doublent tous les 18 mois !!!

Nous avons maintenant défini (au moins partiellement mais suffisamment pour en comprendre les grands mécanismes) les différentes pièces de notre " puzzle ".

Passons maintenant à la phase du montage et de l'assemblage.

II.2. CONSTRUCTION DU " PUZZLE "

Le problème que nous nous posons maintenant est le suivant :

comment associer les composants que nous venons de présenter pour obtenir une architecture et un fonctionnement cohérent de l'ensemble?

De façon très concrète :

- une plaque de circuit imprimé est indispensable (ou tout autre procédé équivalent),
- on reliera, par des pistes de cuivre, les **sorties adresses** du microprocesseur aux **entrées adresses** des mémoires et des composants périphériques, soit tous les **A₀** ensemble, tous les **A₁** ensemble ...

On définit ainsi ce qu'on appelle le **BUS D'ADRESSES (ADDRESS BUS)**.

Par définition, c'est **le chemin électrique et physique** sur lequel les adresses seront **véhiculées**.
(et donc physiquement en **binaire** car les **A_i** valent 0 ou 1).

On voit que ce bus d'adresses est **MONO DIRECTIONNEL**, puisque les adresses circuleront toujours dans le même sens sur ce bus : du microprocesseur **vers** l'extérieur.

Notons que dans ce cas, il s'agit d'un **bus d'adresses externe** aux composants, en opposition au **bus d'adresses interne** qui existe aussi mais à l'intérieur des composants sous forme de pistes de semi-conducteurs,

- on reliera, par des pistes de cuivre, les **entrées/sorties données** du microprocesseur aux **entrées/sorties données** des mémoires et des composants périphériques, soit tous les **D₀** ensemble, tous les **D₁** ensemble ...

On définit ainsi ce qu'on appelle le **BUS DE DONNES (DATA BUS)**.

Par définition, c'est **le chemin électrique et physique** sur lequel les données seront **échangées**.
(et donc physiquement en **binaire** car les **D_i** valent 0 ou 1).

On voit que ce bus de données est **BI DIRECTIONNEL**, puisque les données circuleront tantôt dans un sens, tantôt dans l'autre sur ce bus : du microprocesseur **vers** l'extérieur pour une **écriture** dans une case mémoire ou dans un registre interne d'un circuit périphérique et de l'extérieur **vers** le microprocesseur pour une **lecture** d'une case mémoire ou d'un registre interne d'un circuit périphérique.

Notons que dans ce cas aussi il s'agit d'un **bus de données externe** aux composants, en opposition au **bus de données interne** qui existe aussi mais à l'intérieur des composants sous forme de pistes de semi-conducteurs,

- on reliera par des pistes de cuivre, la **sortie de commande lecture/écriture** du microprocesseur à **l'entrée** correspondante des mémoires et des composants périphériques.

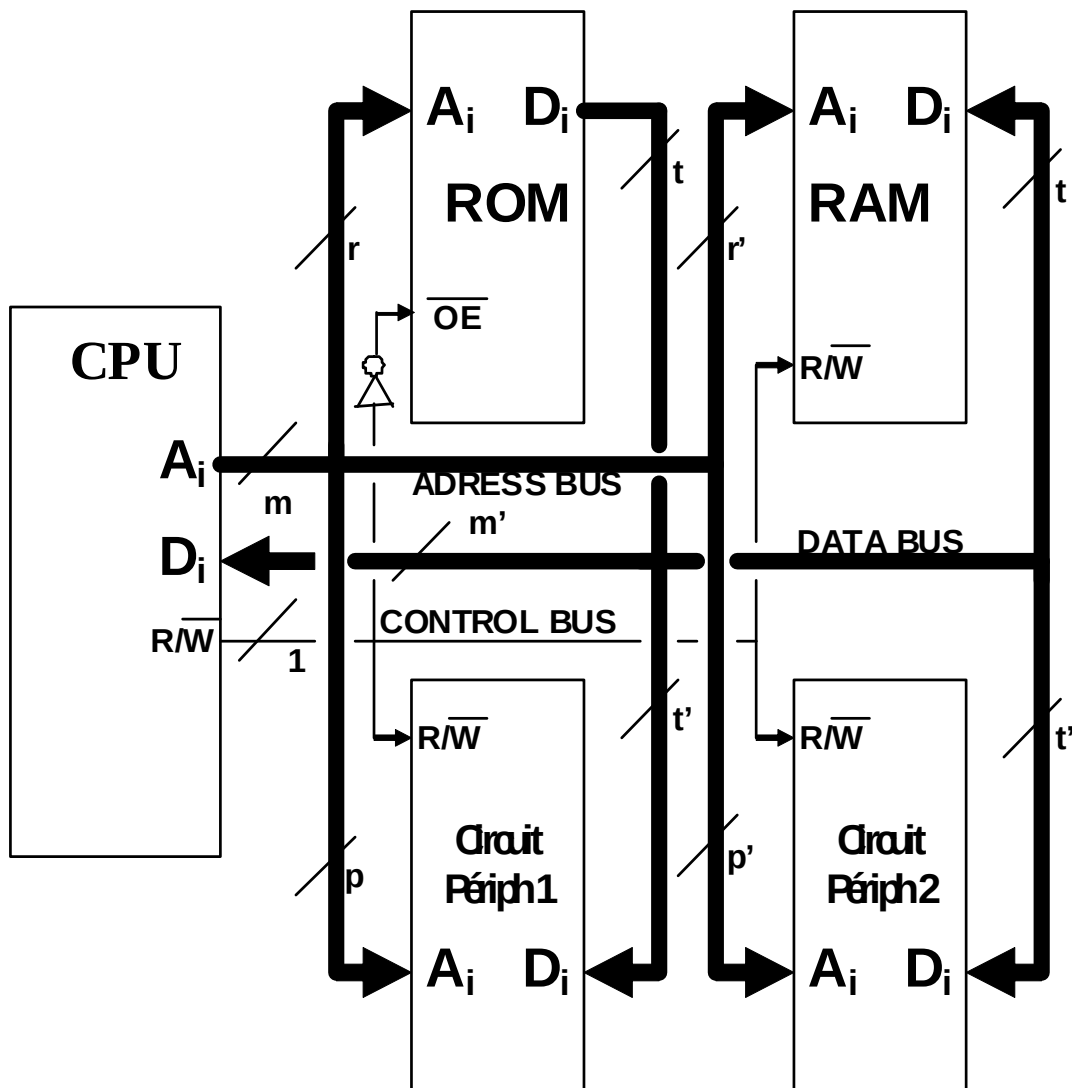
On définit ainsi ce qu'on appelle le **BUS DE CONTRÔLE (CONTROL BUS)**.

Par définition, c'est **le chemin électrique et physique** où les signaux de contrôle du système sont envoyés.

Là aussi, il s'agit d'un **bus de contrôle externe**.

Notons, et on s'en doute bien, qu'en pratique ce bus n'aura pas qu'un seul fil comme ici ...

Ce qui nous donne le synoptique (**très**) simplifié de la figure II.9. suivante :



Architecture type dite de Von Neuman

Figure II.9.

Pourquoi **très** simplifié ?

Tout simplement parce que nous avons fait l'impasse sur un certain nombre de problèmes que nous allons maintenant aborder.

Nous en détaillerons au moins **quatre**. Puis, nous essaierons de les résoudre pour arriver à une nouvelle proposition d'architecture, qui ne sera, elle aussi hélas, pas complète, mais en tout cas sera plus proche des architectures réelles.

1^{er} problème relatif au DATA BUS :

On peut sans problème relier les broches A_i (les broches A_0 ensemble, les broches A_1 ensemble ...) : en effet chaque A_i est soit une **sortie** du microprocesseur, soit une **entrée** des mémoires ou des circuits périphériques. Electriquement parlant, cela ne pose aucun problème, car on peut relier une sortie sur plusieurs entrées, c'est même pour cela que c'est prévu.
(avec cependant la petite réserve du 3^{ème} problème qu'on verra plus loin ...).

Par contre, pour le bus de données la situation est différente : en effet chaque broche D_i est fondamentalement une broche d'**entrée/sortie**, et donc peut devenir à tout moment une **sortie**.

Et en électronique, il faut prendre des précautions lorsque l'on relie deux (ou plusieurs) sorties entre elles. Pourquoi ?

Pour comprendre le risque de cette manipulation, il faut se rappeler la structure d'un étage de sortie de la très grande majorité des composants en électronique numérique : le « **totem pôle** » représenté sur la figure II.10. ci-dessous :

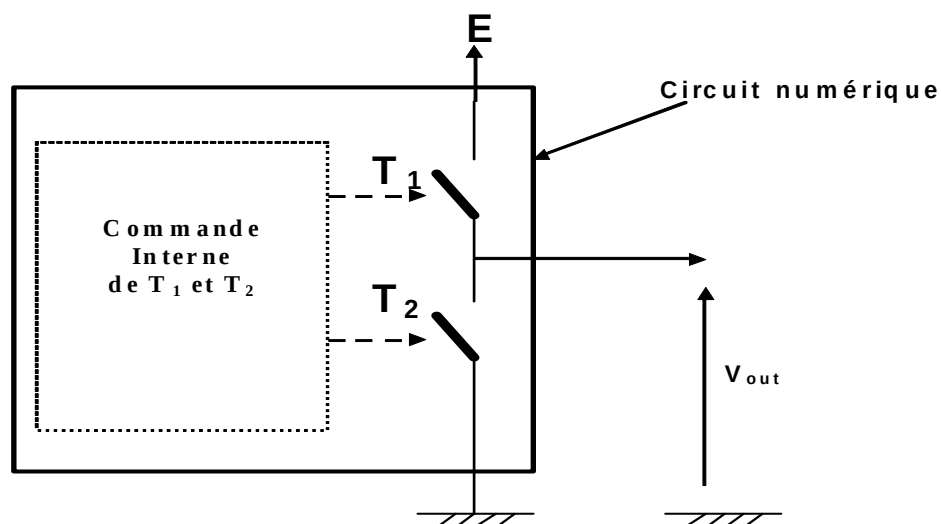


Figure II.10.

T_1 et T_2 sont 2 transistors qui fonctionnent comme des **interrupteurs idéaux**. La tension sur la broche de sortie V_{out} dépend de l'état de ces deux transistors, suivant le tableau ci-dessous :

Etat de T_1	Etat de T_2	V_{out}
Passant = fermé = ON	Bloqué = ouvert = OFF	E
Bloqué = ouvert = OFF	Passant = fermé = ON	0
Bloqué = ouvert = OFF	Bloqué = ouvert = OFF	High Z = haute impédance
Passant = fermé = ON	Passant = fermé = ON	Interdit car E en court-circuit

Si maintenant, on relie 2 sorties de type « totem-pôle » entre elles, on obtient la configuration suivante, résumé sur la figure II.11. :

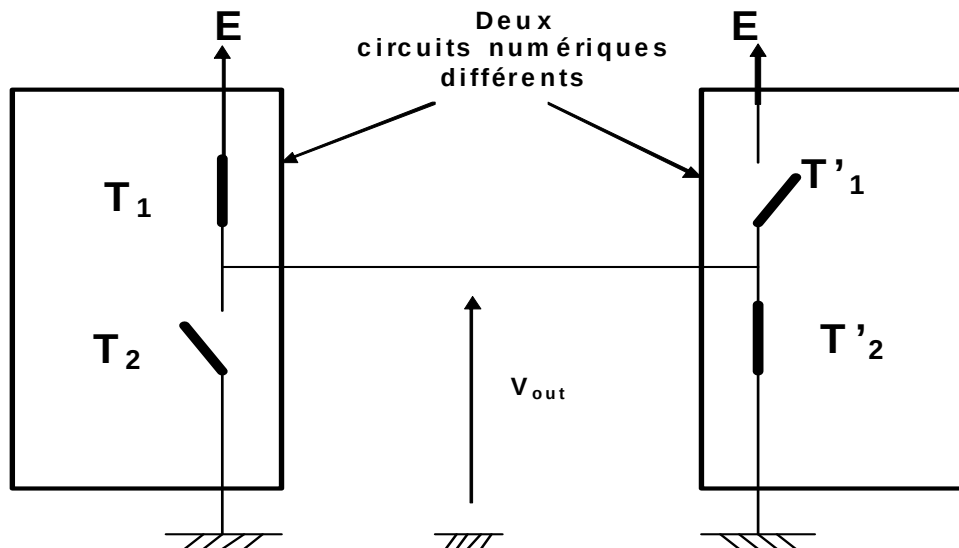


Figure II.11.

Un problème se posera par exemple lorsque le circuit de gauche sera en sortie au niveau **haut** (T_1 **ON** et T_2 **OFF**) et celui de droite sera au niveau **bas** (T'_1 **OFF** et T'_2 **ON**) : l'alimentation E sera en court-circuit via T_1 et T'_2 (comme sur la figure II.11.) ce qui risque de faire circuler un courant important dans ces 2 transistors et d'endommager les composants.

Une autre façon de voir les choses mais à un niveau logique :
si une sortie qui vaut 5 V est reliée à une sortie qui vaut 0 V, quelle sera la valeur logique résultante ?

Réponse : impossible à savoir !!!

Comment résoudre cette difficulté dans le cas du DATA BUS ?

Il suffit d'utiliser la troisième ligne du tableau d'état précédent, celle où les 2 transistors sont tous les deux bloqués, état appelé HAUTE IMPEDANCE. (HIGH Z).

Ainsi, si on relie toutes les broches D_i ensemble (supposons qu'il y en ait n au total (c'est-à-dire n broches D_0 , n broches D_1 , ...) : le D_i du microprocesseur, plus ceux des mémoires et des circuits périphériques), il faudra s'assurer à tout instant que **n-1** broches D_i soient en **haute impédance** et **une seule** broche D_i soit en **sortie**.

Ceci sera fait grâce à une commande de type $\overline{OE}$ (pour $\overline{OUTPUT\ ENABLE}$ c'est-à-dire validation de sortie), existante sur tous les composants relié au bus de données : à tout instant, un seul composant aura son entrée $\overline{OE}$ au niveau bas (ce composant aura alors ses broches D_i en sortie), les autres $\overline{OE}$ étant au niveau haut (leurs broches D_i seront en haute impédance).

2^{ème} problème relatif à l'ADDRESS BUS :

Ce problème va nous permettre d'introduire la notion **importante** de **DECODAGE D'ADRESSE** .

Voyons cela sur un exemple simple.

On dispose d'un microprocesseur ayant **16** broches d'adresses, donc un bus d'adresses de **A₀** à **A₁₅**.
Il peut donc adresser au maximum $2^{16} = 65\,536 = 64\text{ K}$ cases mémoire différentes.

Supposons que nous souhaitions implanter ce microprocesseur sur une carte avec une R.A.M. de 32Ko (32 768 octets) et une R.O.M. de la même capacité, pour pouvoir bénéficier du maximum de mémoire adressable par le microprocesseur (32 Ko + 32 Ko = 64 Ko !!!).

Chacun de ces deux circuits de mémoire possédera donc, comme on l'a vu :

- une case numéro **0**,
- une case numéro **1**,
- ...
- ...
- ...
- une case numéro **32767**.

Lorsque le microprocesseur voudra lire le contenu de la case d'adresse 10 (par exemple), va t'il accéder à la RAM ou à la ROM ?

avec notre système **très** simplifié précédent, il faut répondre :

" les deux, mon capitaine ! "

Cette situation n'est pas acceptable. On a ce qu'on appelle un **conflit de bus**.

C'est pourquoi, il faut pouvoir **différencier de façon certaine** les deux circuits de mémoire **différents**.

Pour cette raison, chaque circuit (mémoires ou périphériques) qui intervient dans une architecture à base de microprocesseur et qui à un moment ou un autre doit échanger des informations avec le microprocesseur possède une entrée (ou parfois plusieurs) notée souvent :

- $\overline{\text{CS}}$ (ou CS) comme **CHIP SELECT** ou **SELECTION DU COMPOSANT**,

ou

- $\overline{\text{CE}}$ (ou CE) comme **CHIP ENABLE** ou **AUTORISATION DU COMPOSANT**.

C'est cette entrée $\overline{\text{CS}}$, qui à l'état haut met les broches de données du composant en haute-impédance, un peu comme si le composant était momentanément déconnecté du système. Seul le composant qui aura son entrée $\overline{\text{CS}}$ au niveau bas aura ses broches **D_i** effectivement reliées au bus de données du système. (un seul $\overline{\text{CS}}$ doit être à 0 (celui du circuit adressé) les autres sont à 1, car si plusieurs entrées $\overline{\text{CS}}$ étaient à 0 au même moment, on retomberait sur la situation précédente avec un conflit sur le bus de données).

**La voilà la fameuse entrée qui manquait à nos mémoires ou à nos circuits périphériques !!!
(vous pouvez revoir la documentation de la mémoire 2732 page 20 pour vous en convaincre ...)**

La technique utilisée pour commander cette entrée $\overline{CS}$ fonctionne en trois temps (ou 3 étapes) :

Etape 1 :

Cette étape, **théorique**, se fait « sur le papier ». Elle consiste à définir la **CARTOGRAHIE MEMOIRE** (ou **MEMORY MAPPING** ou **CARTE MEMOIRE**) du système.

Ce mapping consiste à partager la capacité totale adressable par le microprocesseur en **différentes** zones (**qui ne se « chevauchent » pas**) et ensuite d'affecter les composants à chacune de ces zones.

Dans notre exemple on peut réaliser le mapping fourni sur la figure II.12. ci-dessous :

- affecter la **R.A.M.** aux **adresses absolues** comprises entre **0 (\$0000)** et **32 767 (\$7FFF)** (adresses **basses** du mapping),

(ainsi la **case 0** de la RAM correspondra à l'adresse absolue **\$0000**, sa case **1** à l'adresse absolue **\$0001**, et sa dernière case (la **32767^{ème}**) à l'adresse absolues **\$7FFF**).

- affecter la **R.O.M.** aux **adresses absolues** comprises entre **32 768 (\$8000)** et **65 535 (\$FFFF)** (adresses **hautes** du mapping).

(ainsi la **case 0** de la ROM correspondra à l'adresse absolue **\$8000** (soit la 32768^{ème} du mapping), sa case **1** à l'adresse absolue **\$8001** (soit la 32769^{ème} du mapping) et sa dernière case (la **32767^{ème}**) à l'adresse absolues **\$FFFF** (soit la 65535^{ème} du mapping).

($\equiv$ ainsi 2 personnes **différentes** ont chacune leur **propre** adresse mail : **OK**)

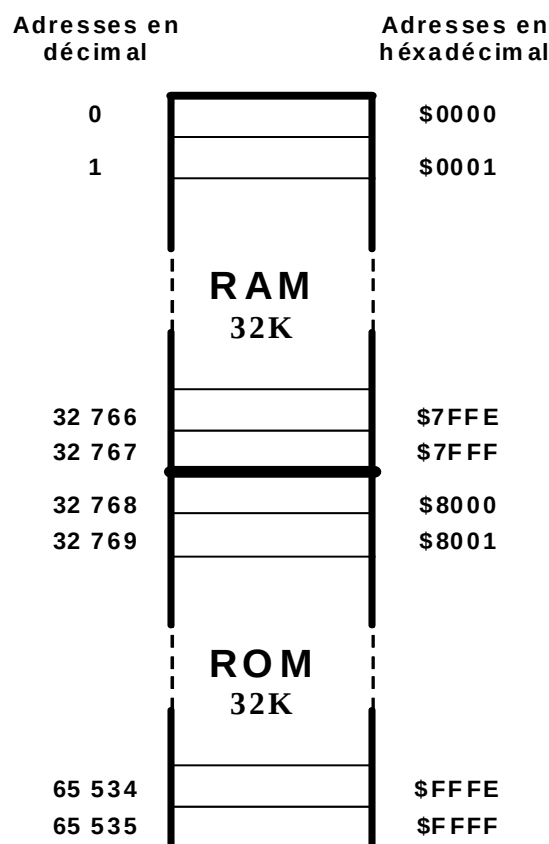


Figure II.12.

Remarque :

A titre indicatif, lorsqu'il n'y a pas de décodage comme dans notre système simplifié précédent de la figure II.9., le mapping aurait été celui de la figure II.13. ci-dessous :

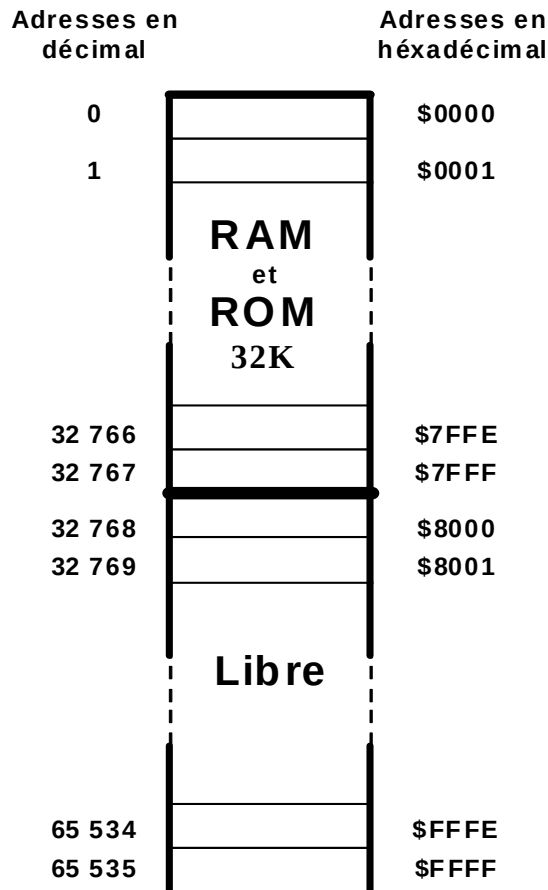


Figure II.13.

Et on voit que dans ce cas là, on ne peut pas différencier la RAM de la ROM : il y aura bien conflit sur le bus de données (impossible de différencier les cases mémoire de la RAM et celles de la ROM !!!) (il y a 2 cases d'adresses absolues \$0000, 2 cases d'adresses absolues \$0001 ...).

(≡ ici 2 personnes **différentes** ont la **même** adresse mail : **KO**)

Par contre, le mapping que l'on a choisi en affectant la RAM aux adresses basses et la ROM aux adresses hautes n'est pas le seul.

C'est effectivement le **plus simple**. Mais on aurait tout aussi bien pu faire le découpage donné sur la figure II.14. (4 zones différentes de 16 Ko au lieu de 2 zones de 32 Ko).

Adresses en décimal		Adresses en hexadécimal
0		\$0000
1		\$0001
	RAM 16K	
16 382		\$3FFE
16 383		\$3FFF
16 384		\$4000
16 385		\$4001
	ROM 16K	
32 766		\$7FFE
32 767		\$7FFF
32 768		\$8000
32 769		\$8001
	RAM 16K	
49 150		\$BFFE
49 151		\$BFFF
49 152		\$C000
49 153		\$C001
	ROM 16K	
65 534		\$FFFE
65 635		\$FFFF

Figure II.14.

Le concepteur d'un système est donc à priori libre de choisir le mapping qu'il veut, en fonction des composants qu'il a choisis.

Les circuits périphériques doivent eux aussi se trouver dans le mapping à des adresses où il n'y a pas de cases mémoires. Ils auront donc aussi une adresse absolue (ou plus généralement une plage d'adresses contiguës s'ils possèdent plusieurs registres internes).

Dans certains cas, en fonction du microprocesseur, on sera obligé de mettre la ROM à une adresse particulière (après un RESET ou à la mise sous tension, le microprocesseur recherche toujours la première instruction à la même adresse imposée par le constructeur du microprocesseur : il faudra donc placer de la ROM à cette adresse, car si c'est de la RAM, il y aura n'importe quoi à cette adresse et le microprocesseur ne trouvera pas la première instruction à exécuter : le système plantera à tous les coups...).

Tous les systèmes auront donc leur propre cartographie mémoire, qu'il faudra respecter pour accéder aux différents composants (mémoires et circuits périphériques).

Etape 2 :

Il s'agit, dans cette étape de trouver les **équations logiques** permettant de trouver les signaux $\overline{CS}$ en fonction des valeurs des A_i .

En pratique, il vaut mieux exprimer les adresses en notation hexadécimale, ou mieux en binaire.

Nous pouvons réécrire le mapping de la figure II.12. d'une autre façon, en développant l'**ADDRESS BUS** en binaire et en écrivant pour chaque zone, la première (début) et la dernière adresse (fin), comme dans le tableau ci-dessous :

Circuit	Hexadécimal	A_{15}	A_{14}	A_{13}	A_{12}	A_{11}	A_{10}	A_9	A_8	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0
RAM	Début : \$0000	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	à	0	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X
	Fin : \$7FFF	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
ROM	Début : \$8000	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	à	1	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X
	Fin : \$FFFF	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

En examinant ce tableau, il apparaît clairement que le seul bit significatif pour différencier les 2 circuits de mémoire est le bit A_{15} , les autres bits du bus d'adresses (A_{14} à A_0) pouvant prendre toutes les valeurs pour la RAM et pour la ROM.

Ainsi :

- pour la R.A.M. : $A_{15} = 0$, quelque soient les autres adresses,
- pour la R.O.M. : $A_{15} = 1$, quelque soient les autres adresses.

C'est ce signal que nous utiliserons pour le décodage, de la façon suivante :

- $\overline{CS}_{RAM} = A_{15}$ et donc A_{15} connecté au $\overline{CS}$ de la R.A.M. (actif au niveau bas),
- $\overline{CS}_{ROM} = \overline{A_{15}}$ et donc $\overline{A_{15}}$ connecté au $\overline{CS}$ de la R.O.M. (actif au niveau bas).

Etape 3 :

On procède enfin à la **réalisation électronique** du **DECODAGE D'ADRESSES** :

c'est l'ensemble des composants qui concourent à la différenciation totale des circuits contenant des informations échangeables avec le microprocesseur (mémoires et circuits périphériques).

Ici, le cas est très simple puisqu'un simple inverseur suffit pour réaliser ce décodage d'adresses comme on peut le voir sur la figure II.15. :

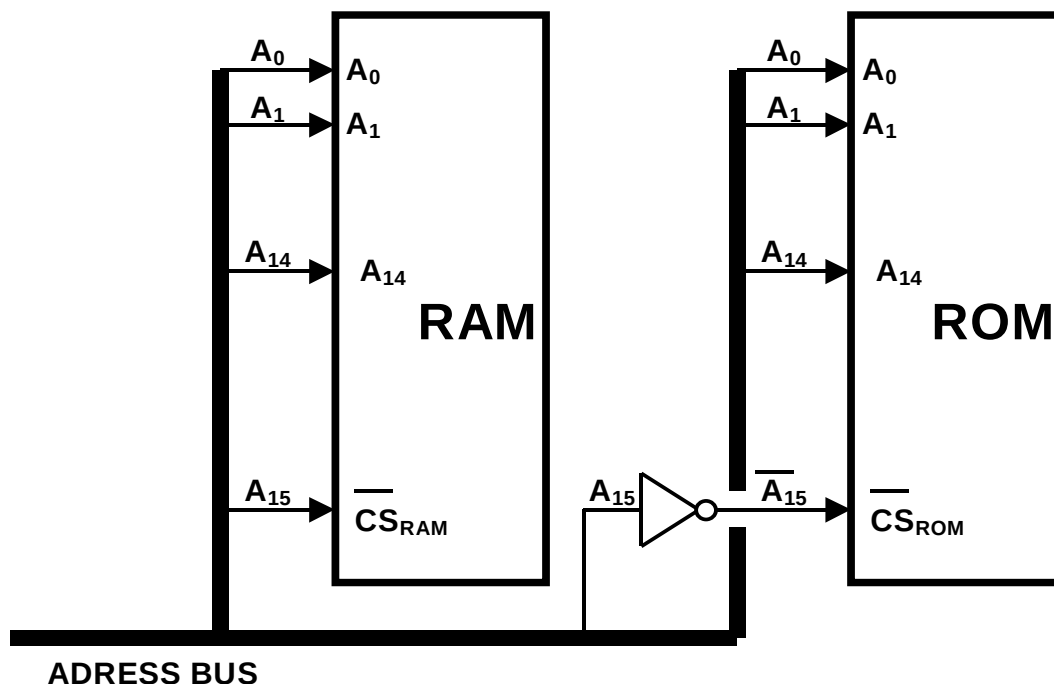


Figure II.15.

Au final :

- la ligne A_{15} permet de choisir soit la RAM (quand elle vaut 0) soit la ROM (quand elle vaut 1),
- et les lignes A_0 à A_{14} permettent de choisir une case dans la mémoire sélectionnée par A_{15} .

De façon très générale, dans une électronique de décodage :

- les bits de **poids forts du bus d'adresses** permettent de sélectionner un **composant**,
- les bits de **poids faibles du bus d'adresses** permettent de sélectionner une **case** à l'intérieur du composant sélectionné par les bits de poids forts.

Si nous reprenons l'exemple de notre case 10 (\$0A) de la RAM et de la ROM, on aura :

- adresse absolue de la case 10 de la RAM : **\$000A**
- adresse absolue de la case 10 de la ROM : **\$800A**

Dans la pratique, une électronique de décodage pourra être plus complexe qu'ici :

- le nombre de composants à décoder est plus grand que deux (plusieurs boîtiers de mémoire, plusieurs circuits périphériques ...),
- d'autres signaux du bus de contrôle liés au fonctionnement du bus d'adresses ou à la synchronisation du système peuvent intervenir dans l'équation logique permettant de déterminer les $\overline{CS}$...

On utilisera, au lieu de simples portes logiques comme ici, des décodeurs ou mieux encore des P.A.L. (**P**rogrammable **A**rray **L**ogic) qui sont des composants intégrant des fonctions logiques (combinatoires et/ou séquentielles) prêtes à être assemblées entre elles pour réaliser une équation logique (un peu comme dans les composants XILINX mais en plus simple).

3^{ème} problème relatif à tous les BUS :

On peut voir sur le synoptique simplifié de la figure II.9., que plusieurs circuits sont connectés sur le même signal (par exemple tous les **D₀** sont connectés ensemble, tous les **D₁** sont connectés ensemble... tous les **A₀** sont connectés ensemble ...).

Ce type de configuration peut entraîner des problèmes de compatibilité au niveau des tensions électriques.

En effet, si on tire trop de courant sur une sortie à l'état haut pour alimenter plusieurs entrées différentes, la tension de cette sortie peut diminuer (cf. théorème de Thévenin de l'électronique analogique et de l'impédance de sortie des composants non parfaitement nulle) et peut passer au-dessous du niveau minimal garanti pour être interprété comme un niveau haut par les entrées qui sont branchées sur cette sortie.

On aura ainsi un grave dysfonctionnement.

Pour cette raison, il existe des circuits intégrés spéciaux, appelés **AMPLIFICATEUR DE BUS**.

On les place en série sur le bus et ils évitent le problème précédent. (en fait ils ne changent pas le niveau logique, mais ils sont prévus pour être capable de fournir en sortie un courant important sans chute notable de la tension de sortie).

- sur un **ADDRESS BUS**, les amplificateurs de bus seront **monodirectionnels** :
l'adresse est toujours générée par le microprocesseur vers l'extérieur.
exemple : le 74 HC 244 (voir pages suivantes).

- sur un **DATA BUS**, les amplificateurs de bus seront **bidirectionnels** :
les données circulent de l'extérieur vers le microprocesseur pendant les phases de lecture et du microprocesseur vers l'extérieur pendant les phases d'écriture.
exemple : le 74 HC 245 ou le 8286 (voir pages suivantes).

Exercice :

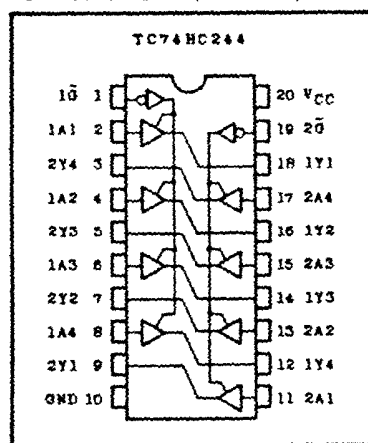
Sur le **74HC245** et sur le **8286**, comment se fait la sélection du sens de l'amplification ?

- sur un **CONTROL BUS**, les amplificateurs de bus seront la plupart du temps **monodirectionnels**.

TC74HC244P OCTAL BUS BUFFER WITH INVERTED 3-STATE OUTPUTS

The TC74HC240 (see page 74), TC74HC241 (see page 76) and TC74HC244 are high speed CMOS OCTAL BUS BUFFERS fabricated with silicon CMOS technology. They achieve the high speed operation similar to equivalent LSTTL while maintaining the CMOS low power dissipation. The designer has a choice of selected combinations of inverting and non-inverting outputs, symmetrical $\bar{G}$ (active-low output control) inputs, and complementary G and $\bar{G}$ inputs. Each control input govern four BUS BUFFERS. These devices are designated to be used with 3-state memory address drivers, etc. All inputs are equipped with protection circuits against static discharge or transient excess voltage.

PIN ASSIGNMENT (TOP VIEW)



TRUTH TABLE

INPUTS		OUTPUTS	
$\bar{G}$	A_n	Y_n	
L	L	L	
L	H	H	
H	X	Z	

X : Don't care
Z : High Impedance

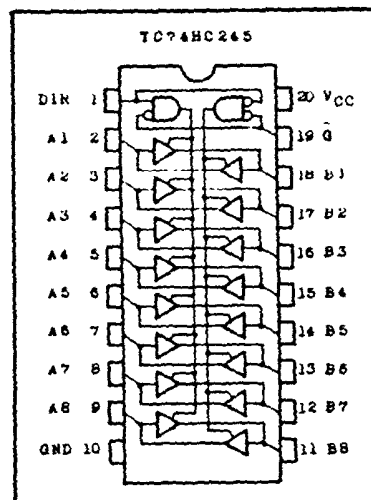
AC ELECTRICAL CHARACTERISTICS ($C_L = 50$ pF, Input $t_r = t_f = 6$ ns)

PARAMETER	SYMBOL	V_{CC}	$T_a = 25^\circ C$			$T_a = -40 \sim 85^\circ C$		UNIT
			MIN.	TYP.	MAX.	MIN.	MAX.	
Propagation Delay Time	t_{PLH}	2.0	—	50	110	—	135	ns
	t_{PHL}	4.5	—	14	22	—	27	
		6.0	—	12	19	—	23	
Input Capacitance	C_{IN}		—	5	10	—	10	pF
Output Capacitance	C_{OUT}		—	10	—	—	—	pF
Power Dissipation Capacitance	C_{PD}		—	40	—	—	—	pF

TC74HC245P 3-STATE, NON-INVERTING OCTAL BUS TRANSCEIVER

The TC74HC245, TC74HC640 (see page 126) and TC74HC643 (see page 127) utilize silicon gate C²MOS technology to achieve operating speed equivalent to LSTTL parts. Along with the low power dissipation and high noise immunity of standard C²MOS integrated circuit, it possesses the driving capability of 15 LSTTL loads. These ICs are intended for two-way asynchronous communication between data buses, and the direction of data transmission is determined by DIR input. The enable input ($\bar{G}$) can be used to disable the device so that the buses are effectively isolated. All inputs are equipped with protection circuits against static discharge or transient excess voltage.

PIN ASSIGNMENT



TRUTH TABLE

INPUT		FUNCTION		OUTPUT
$\bar{G}$	DIR	A BUS	B BUS	HC245
L	L	OUTPUT	INPUT	A = B
L	H	INPUT	OUTPUT	B = A
H	*	Z		Z

*: "H" or "L" Z: High Impedance

AC ELECTRICAL CHARACTERISTICS ($C_L = 50$ pF, Input $t_r = t_f = 6$ ns)

PARAMETER	SYMBOL	V _{CC}	T _a = 25 °C			T _a = -40 ~ 85 °C		UNIT
			MIN.	TYP.	MAX.	MIN.	MAX.	
Propagation Delay Time	t _{PLH}	2.0	—	50	110	—	135	ns
		4.5	—	13	22	—	27	
	t _{PHL}	6.0	—	11	19	—	23	
Input Capacitance	C _{IN}		—	5	10	—	10	pF
Bus Input Capacitance	C _{I/O}		—	13		—		pF
Power Dissipation Capacitance	C _{PD}		—	33	—	—	—	pF

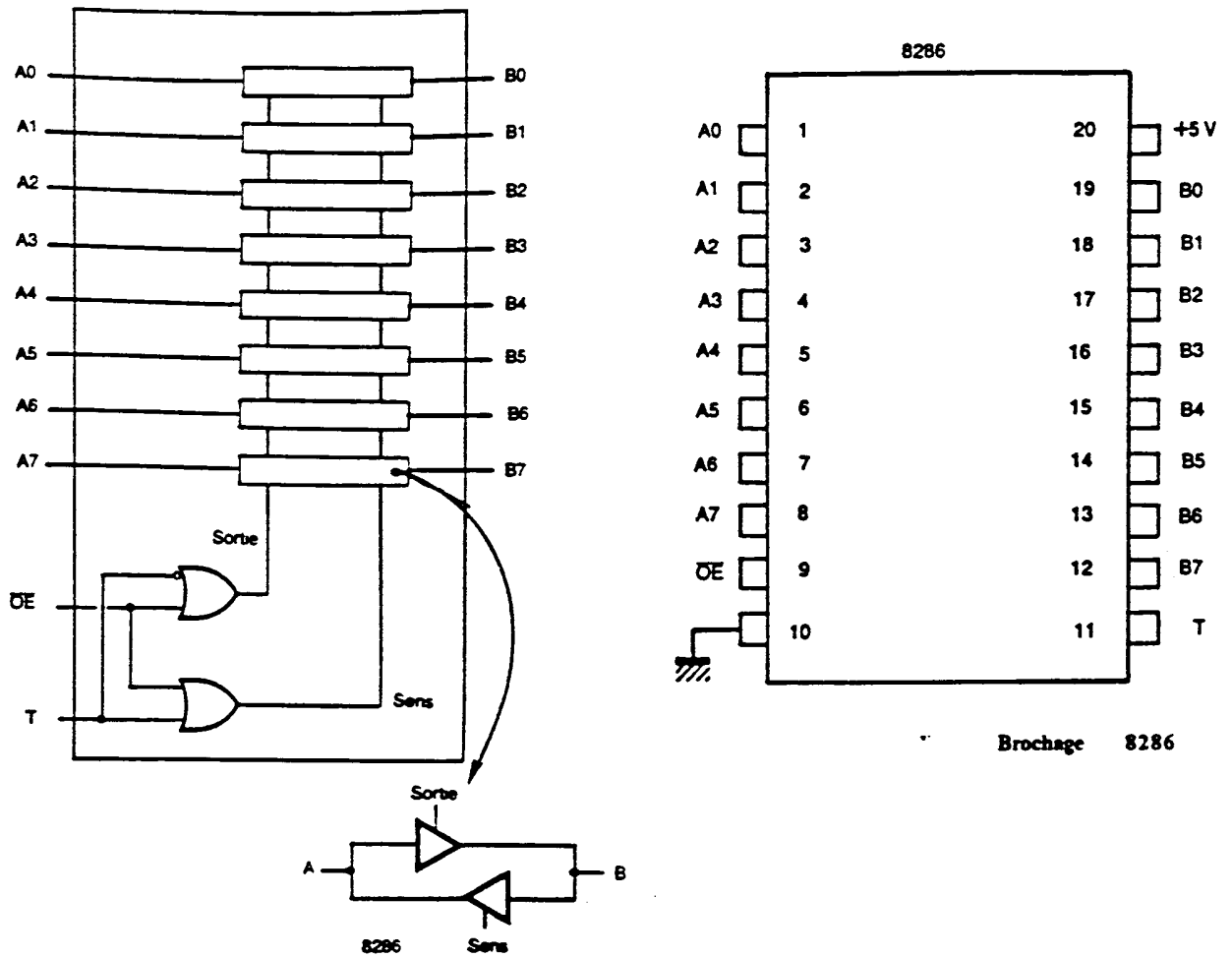


TABLE D'ETAT DU 8286

Niveaux sur		Lignes A	Lignes B	Signal sur la sortie réelle A ou B
OE	T			
Bas	Haut	Entrées	Sorties	Celui de l'entrée
Bas	Bas	Sorties	Entrées	Celui de l'entrée
Haut	Haut ou bas	Entrées ou sorties	Entrées ou sorties	Etat haute impédance

4^{ème} problème relatif au système complet :

On rencontrera dans la pratique rarement des architectures aussi simples.

Les systèmes réels sont plus complexes.

Ils présentent notamment les complémentarités suivantes :

- **rajout d'une logique d'interruption,**

(pour augmenter dans certains cas la vitesse de traitements d'événements extérieurs ou pour décharger le microprocesseur de certaines tâches),

- **un bus de contrôle plus complet,**

(pour fiabiliser le fonctionnement et/ou faciliter le couplage entre plusieurs microprocesseurs),

- **multiplexage (temporel) possible entre les adresses et les données** (un même broche véhiculera tantôt un bit de donnée, tantôt un bit d'adresse) ce qui permet de diminuer le nombre de broches du microprocesseur, mais nécessite une électronique extérieure supplémentaire de **démultiplexage**,

- **l'existence d'un BUS D'EXTENSION (EXTENSION BUS)**

(où sont regroupés, la plupart du temps sur un ou plusieurs connecteurs adéquats tous des signaux utiles (ADDRESS BUS, DATA BUS et CONTROL BUS) pour pouvoir éventuellement utiliser les ressources du système de base en rajoutant d'autres cartes électroniques.

Par exemple bus d'extension du P.C. : anciennement **BUS ISA** mais maintenant **BUS PCI**.

- ...

Voilà donc le synoptique de la figure II.16. plus proche de la réalité et prenant en compte la plupart des modifications et des améliorations précédentes.

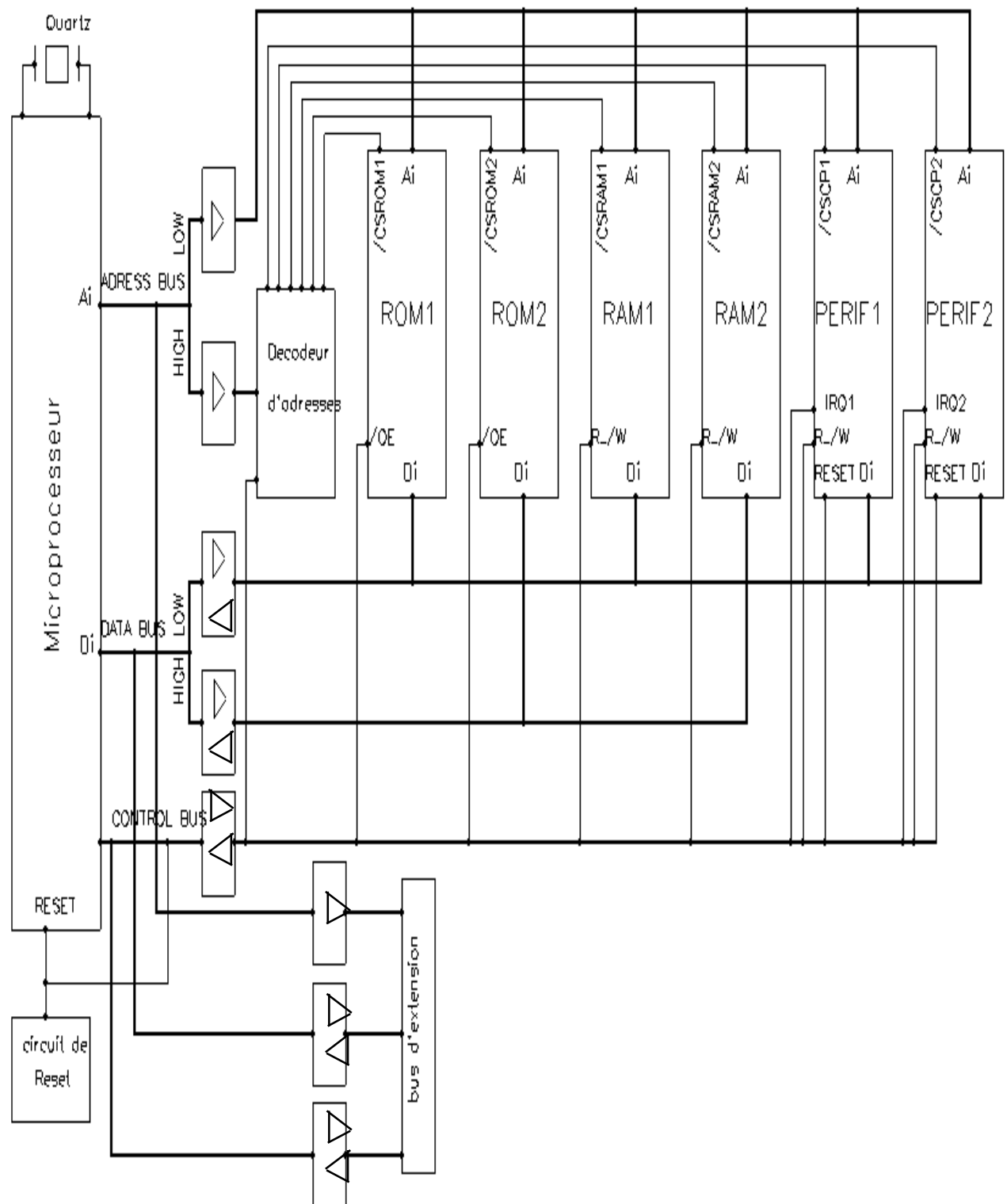


Figure II.16.

III. A L'INTERIEUR DU MICROPROCESSEUR

Je dirai qu'il s'agit : de la passerelle entre le **HARD** et le **SOFT**,

ou

" **ce que le HARD fournit au SOFT** ".

Nous serons ici à la frontière entre ces deux domaines.

C'est donc le moment de rentrer " à l'intérieur " du microprocesseur pour y définir une architecture interne cohérente avec le type de broches que nous lui avons " intuitivement " affectées (vu de l'extérieur). Et de voir ainsi comment le microprocesseur commande ses différents bus.

A priori, et sans prendre de gros risques, on peut supposer qu'il y aura un certain nombre de registres à l'intérieur de tout microprocesseur.

Seront-ils tous équivalents ? On entend par là de savoir s'ils seront :

- **spécifiques** à telle ou telle tâche,

ou au contraire

- **banalisés** c'est-à-dire entièrement **interchangeables** et utilisables pour n'importe quel travail.

Cette deuxième possibilité semble plus séduisante car elle simplifie la vie du programmeur qui n'a plus à se rappeler que tel registre sert dans tel cas alors que tel autre registre ne peut pas être utilisé dans cette configuration.

Et il est vrai que toute l'évolution des microprocesseurs tend vers cette structure idéale, où tous les registres seraient **BANALISES** (pour devenir des « **GENERAL PURPOSE REGISTERS** » c'est-à-dire des registres à usage général).

Il restera malgré tout deux différences difficilement contournables :

- certains registres du microprocesseur servent intrinsèquement à son **fonctionnement propre** (leurs principes de fonctionnement est souvent « **transparent** » pour le programmeur, mais à notre niveau de présentation, il est important de bien le comprendre, car c'est la clef pour savoir comment fonctionne un microprocesseur) alors que d'autres sont de **simples registres de travail** et de **stockage temporaire** d'information liée au traitement effectué,

- certains registres contiennent des **données** (des **variables** : par exemple quand on déclare un **int** , un **char** ou un **float** en langage C, ce sera par ces registres de données qu'il transitera dans le microprocesseur pour y être traité) ...
et d'autres contiennent des **adresses** (des **pointeurs** : par exemple quand on déclare un **int ***, un **char *** ou un **float *** en C, ce sera par ces registres d'adresses qu'il transitera dans le microprocesseur pour y être traité).

Intéressons nous dans un premier temps aux registres liés **intimement** au fonctionnement du microprocesseur. Ils sont **au moins** au nombre de trois (parfois il y en a plus sur certains microprocesseurs) et on les retrouvera (plus ou moins sous cette forme) dans tous les microprocesseurs ou tous les microcontrôleurs :

- **LE PROGRAM COUNTER : PC (ou compteur de programme),**

- **LE STATUS REGISTER : SR (ou registre d'état).**

- **LE STACK POINTER : SP (ou pointeur de pile),**

Nous parlerons également dans cette partie du **cycle instruction** et de **trois blocs fonctionnels** que l'on trouvera également dans tout microprocesseur :

- **L'UNITE DE RECHERCHE D'INSTRUCTIONS (FETCH UNIT),**
- **L'UNITE DE DECODAGE D'INSTRUCTIONS (DECOD UNIT),**
- **L'UNITE D'EXECUTION ou UNITE ARITHMETIQUE ET LOGIQUE (U.A.L.)**

III.1. LE PROGRAM COUNTER ET LE SEQUENCEMENT DU PROGRAMME

Nous avons vu dans la première partie, que le microprocesseur devait être capable de rechercher lui-même et de façon quasi-automatique les instructions en mémoire. (la liste du travail à effectuer).

Pour cela, il faut bien qu'il possède un registre spécial qui lui indique, à tout instant, l'endroit où se trouve l'instruction suivante à exécuter. (sinon comment ferait-il pour y accéder ???).

Ce registre est appelé le **PROGRAM COUNTER PC** (ou **COMPTEUR ORDINAL**).

Note : certains constructeurs, comme **Intel** par exemple l'appelle **L'INSTRUCTION POINTER** (ou **POINTEUR D'INSTRUCTIONS**). Mais tous ces termes sont en fait des synonymes ...

A l'initialisation ou après un reset du microprocesseur, le **PC** est automatiquement initialisé (par construction) avec une valeur garantie par le fabricant. Il pointera donc au début toujours vers la même case mémoire (qui sera de la ROM et qui contiendra la toute première instruction à exécuter).

Puis le **PC** est automatiquement et séquentiellement incrémenté (on verra plus loin de combien) pour pointer sur l'instruction suivante etc etc ...

Le PC contient à chaque instant une adresse qui correspond à celle de la prochaine instruction à exécuter.

On dit que le PC *pointe* sur la prochaine instruction.

Pour comprendre le fonctionnement du PC, il faut dire quelques mots sur le cycle instruction.

III.2. LE CYCLE INSTRUCTION

Comment le microprocesseur gère-t-il le contenu de son **PC** ? Voyons cela de façon simplifiée.

Il se déroule ce que l'on appelle un **CYCLE INSTRUCTION** composé de trois phases :

- PHASE 1 : RECHERCHE DE L'INSTRUCTION (INSTRUCTION FETCH) :

Le contenu du **PC** est placé sur le bus d'adresses ce qui permet grâce à la logique de décodage externe de sélectionner la case mémoire correspondante. En parallèle, le microprocesseur commande le signal du bus de contrôle pour indiquer qu'il s'agit d'une opération de lecture mémoire (signal de sortie $R/\overline{W}$ mis à 1, pour signaler une phase de « READ mémoire »).

Puisqu'il s'agit d'une recherche d'instruction, la R.O.M. sélectionnée (celle qui aura son $\overline{CS}$ à 0 et son $\overline{OE}$ (via le signal $R/\overline{W}$) aussi à 0) fournira sur le bus de données le contenu de la case mémoire **pointée** par le **PC**.

Tout ce mécanisme correspond au fonctionnement de **L'UNITE DE RECHERCHE D'INSTRUCTIONS (INSTRUCTION FETCH UNIT)**.

Il ne reste plus au microprocesseur qu'à **lire** (on dit aussi **échantillonner**) cette combinaison présente sur le bus de données.

Remarque 1 :

Les instructions ne sont pas toujours situées en R.O.M.

Par exemple, sur un micro-ordinateur, lorsque l'on charge un programme à partir du disque, ce programme est mis en R.A.M. et s'exécute à partir de la R.A.M.. Ce qui est possible car comme la R.O.M., la R.A.M. peut être accédée par une adresse pour être lue. Dans ce cas, là le PC pointe sur de la R.A.M., mais tout ce qui vient d'être dit sur la phase de recherche reste valable.

De façon analogue, lorsqu'on programmera en TP le microcontrôleur HC11, les programmes seront mis en R.A.M.

Remarque 2 :

En pratique, la recherche d'une instruction ne sera pas toujours constituée par une seule phase de recherche en mémoire. En effet, en fonction de l'instruction, et notamment en fonction des opérandes sur lesquels doit travailler cette instruction, il y aura souvent une autre (ou d'autres recherches) en mémoire. Mais le principe est toujours le même.

Ce qui change, c'est qu'en **fonction de la taille** de l'instruction en mémoire, le PC sera **augmenté** (**incrémenté**) de une, ou deux, ou trois (ou même plus) unités pour pointer ensuite sur l'instruction suivante. (de manière générale, si l'instruction en cours occupe **n** octets en mémoire, le PC sera augmenté de **n**, dans le but de toujours pointer ensuite sur la prochaine instruction en mémoire).

Rassurez-vous, c'est le microprocesseur qui calculera automatiquement cette incrémentation et de façon entièrement transparente pour le programmeur. Heureusement ...

- PHASE 2 : DECODAGE DE L'INSTRUCTION (INSTRUCTION DECODE) :

Suite à la phase de recherche de l'instruction, une combinaison de 1 et de 0 est maintenant présente **dans** le microprocesseur.

Cette combinaison représente le code binaire de l'instruction. On parle aussi de **LANGAGE MACHINE** (on détaillera davantage tout cela dans la dernière partie de ce polycopié, mais sachez tout de même qu'une instruction d'addition aura un certain code, une autre de multiplication aura un autre code : chaque tâche élémentaire a son propre code machine). Mais attention, ici quand on parle d'une instruction élémentaire, cela n'a pas le même sens qu'en langage évolué comme le C : une instruction ou une fonction en C (comme printf par exemple) peut représenter plusieurs centaines ou plusieurs milliers (voire même plus dans certains cas) d'instructions élémentaires ...

Il ne lui reste plus qu'à la **décoder**, c'est-à-dire à l'identifier (la **repérer**) par rapport à l'ensemble des instructions que le microprocesseur sait exécuter.

C'est le rôle d'un **bloc logique interne** au microprocesseur, (et qui existera dans tous les microprocesseurs / microcontrôleurs) appelé le **DECODEUR D'INSTRUCTION (DECOD UNIT)**.

Ce bloc se comporte un peu comme une " **boîte noire** " qui en entrée reçoit la combinaison binaire lue pendant la phase de recherche, et qui fournit en sortie les commandes adéquates et nécessaires à la phase suivante.

Noter que :

- pour un microprocesseur **8 bits** (taille du **DATA BUS**) : un octet correspond à 256 combinaisons différentes, et donc un **potentiel de 256 instructions différentes**,
- pour un microprocesseur **16 bits** (taille du **DATA BUS**) : un mot (= 2 octets) correspond à 65 536 combinaisons différentes, et donc un **potentiel de 65 536 instructions différentes**,
- pour un microprocesseur 32 bits ...

On conçoit ainsi que le jeu d'instructions (**INSTRUCTIONS SET** ou ensemble des tâches élémentaires différentes que peut exécuter le microprocesseur) augmentera en puissance avec l'évolution des microprocesseurs.

On parle de technologie **CISC (COMPLEX INSTRUCTIONS SET COMPUTER** ou jeu d'instructions **complexe**) Mais ce n'est pas toujours un avantage, car en pratique, « trop d'instructions peut tuer l'instruction », et s'il y en a trop, elles ne seront pas toutes utilisées.

On préfère alors en implanter moins, tout en essayant de les optimiser : on parle alors de technologie **RISC (REDUCED INSTRUCTIONS SET COMPUTER** ou jeu d'instructions **réduit**).

Exercice :

Sur quel autre facteur gagne t'on également grâce à cette évolution dans la taille du **DATA BUS** ?

- PHASE 3 : EXECUTION DE L'INSTRUCTION (INSTRUCTION EXECUTION) :

C'est la troisième phase du cycle d'instruction : l'instruction, maintenant décodée, doit pouvoir être **exécutée** par le microprocesseur.

C'est le rôle d'un autre bloc fonctionnel appelé **UNITE ARITHMETIQUE ET LOGIQUE (U.A.L.)**

C'est vraiment **le cœur de la C.P.U.** puisque c'est en son sein que sont, au final, effectués **tous les calculs et traitements**.

Son schéma fonctionnel pourrait se représenter comme sur la figure III.1. ci-dessous :

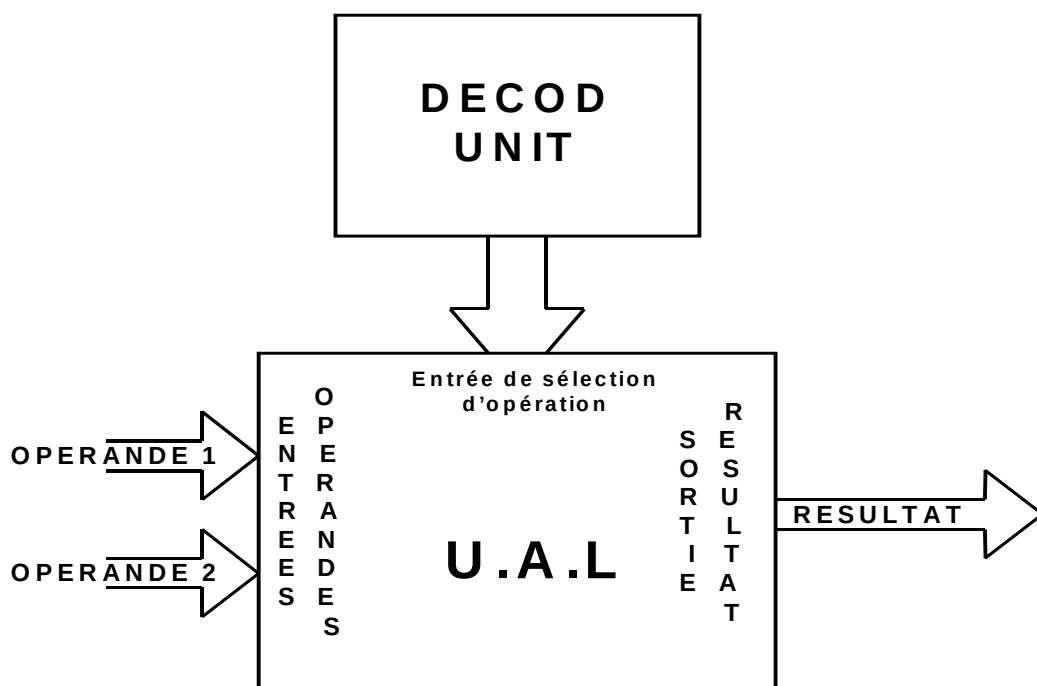


Figure III.1.

Elle reçoit :

- sur ses **entrées de sélection d'opération** : le **code opération** issu des sorties du décodeur d'instructions (phase 2 précédente de décodage de l'instruction),
- sur ses **entrées d'opérandes** : les opérandes à traiter.

Elle fournira sur ses **sorties** le résultat de ce " micro-traitement ".

Ce pourra être toute opération arithmétique (comme l'addition, la soustraction, la multiplication ou la division) ou toute opération logique (and, or, nand, nor, xor, inversion, rotation, décalage, comparaison ...), **un peu comme si elle contenait** tous les circuits intégrés discrets permettant de faire toutes ces opérations et bien plus encore ...

Les possibilités et la puissance de cette U.A.L. augmentent lorsqu'on passera des microprocesseurs 8 bits aux 16 bits puis aux suivants ... pour obtenir au final en quelque sorte une " **SUPER U.A.L.** ".

Souvent, une fois que le traitement a été fait par l'U.A.L., c'est l'instruction suivante qui pourra se charger de gérer ce résultat : le sauvegarder en mémoire, le transférer dans un autre registre interne du microprocesseur ou bien alors s'en servir comme opérande dans le traitement suivant ...

Noter aussi que l'exécution proprement dite peut également être constituée d'un autre accès vers l'extérieur du microprocesseur (pour toutes les instructions de transfert de données, comme par exemple l'instruction qui consisterait à charger le contenu de la case mémoire 42 dans un registre interne du microprocesseur).

On détaillera les grandes familles d'instructions dans les cours suivants.

Donnons un exemple concret pour faire le lien avec le C. Par exemple, quand on écrit en C quelque chose comme :

```
i++ ;
```

on peut envisager la suite des 3 instructions élémentaires suivantes :

- chargement à partir de la case mémoire contenant la variable i dans un registre interne de données du microprocesseur : instruction élémentaire de lecture en mémoire de la case mémoire contenant la variable i et la transférant dans un registre interne du microprocesseur,
- incrémentation de ce registre interne : instruction élémentaire de l'U.A.L. qui rajoute 1 à ce registre interne,
- envoi du contenu de ce registre interne vers la case mémoire stockant la variable i : instruction élémentaire d'écriture en mémoire dans la case mémoire stockant la variable i à partir de ce registre interne.

Ces 3 instructions seront situées en mémoire de programme les unes derrière les autres (dans des cases mémoire qui se suivent) et seront exécutées les unes après les autres pour assurer le « i++ ; » du C.

Il est bon de donner un ordre de grandeur de la **durée du cycle instruction** : c'est le temps qui s'écoule entre le début de la phase de recherche en mémoire de l'instruction et l'instant où cette instruction est totalement exécutée. Il se mesure :

- quelque fois en nombre de **CYCLES BUS** : c'est la durée d'un temps d'accès du microprocesseur vers l'extérieur (**WRITE CYCLE** ou **CYCLE D'ECRITURE**) ou de l'extérieur vers le microprocesseur (**READ CYCLE** ou **CYCLE DE LECTURE**). Certaines instructions, comme on vient de le voir ne nécessitent qu'un cycle bus, d'autres plusieurs (un cycle bus pour la recherche de l'instruction et un (ou plusieurs) cycles bus pour son exécution).
- mais le plus souvent en nombre de **CYCLES HORLOGE** (ou **CLOCK CYCLE** = une **PERIODE D'HORLOGE**) et s'exprime donc **en multiple de la période de l'horloge** du microprocesseur. Il est clair par exemple que dans le cas général, une addition sera par plus rapide qu'une multiplication et correspondra donc à un nombre de cycles horloge plus petit.

Par contre, il faut bien avoir en tête que ces durées sont très faibles. Avec un quartz par exemple à 4 GHz, on a une période d'horloge qui vaut 250 ps !!! Et donc une instruction élémentaire est un phénomène très bref.

Il faut noter cependant que cette présentation du cycle d'instruction a été volontairement simplifiée dans un souci pédagogique.

En pratique, et surtout pour les microprocesseurs les plus récents, **ces trois phases ne seront pas exécutées séquentiellement les unes après les autres.**

En effet, un tel fonctionnement entraîne une perte de temps non négligeable puisque dans ce cas chaque phase doit attendre le résultat de la phase précédente pour démarrer.

Pour utiliser au mieux les potentialités du microprocesseur, ces phases seront exécutées en "**PARALLELE**", grâce à une amélioration de l'architecture interne du microprocesseur. (que nous ne détaillerons pas ici).

Ce type de fonctionnement est appelé en "**PIPE-LINE**", image issue du domaine pétrolier et rappelant les différentes qualités de pétrole que l'on envoie dans les conduits du même nom, sans attendre que la qualité précédente soit arrivée à destination.

Le chronogramme de la figure III.2. illustre la différence entre notre présentation simplifiée et un fonctionnement en **pipe-line**. Notez le gain de temps ...

Légende à utiliser :

Ri : durée de la phase de recherche de l'instruction i,

Di : durée de la phase de décodage de l'instruction i,

Ei : durée de la phase d'exécution de l'instruction i.

WS : durée d'attente (**WAIT STATE**) pour synchroniser les 3 unités (une unité ne peut commencer à fonctionner que si la précédente a fini son travail)

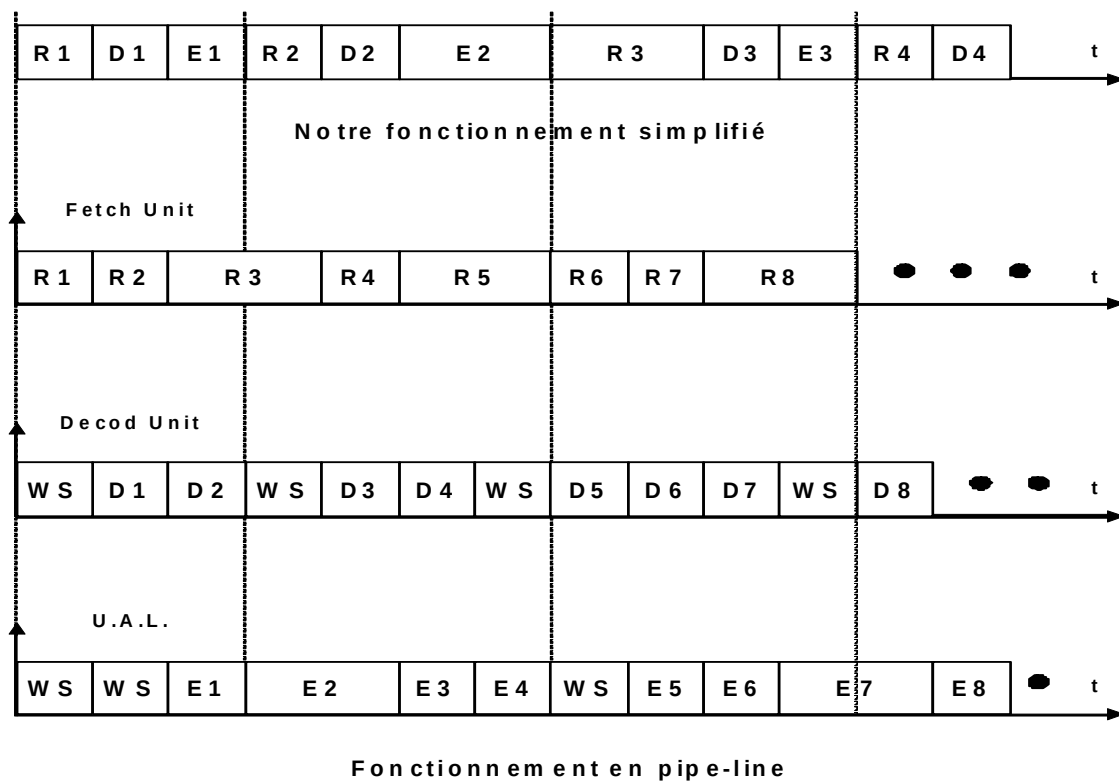


Figure III.2.

Remarque :

Le concept du **CACHE MEMOIRE (CACHE MEMORY)** relève également un peu de la même philosophie (mais en plus complexe) : gagner toujours un peu plus de temps. L'idée, dans ce cas, est de placer dans une RAM **statique (plus chère mais plus rapide)** située à **l'intérieur** même du microprocesseur, les instructions **le plus souvent utilisées** du programme. Au lieu d'aller les chercher dans la mémoire **extérieure (moins chère mais plus lente)**, le microprocesseur les aura ainsi directement à sa disposition. Cette mémoire cache est une mémoire dite **ASSOCIATIVE** du nom de l'algorithme qui permet de détecter les instructions les plus fréquentes pour les placer dans ce cache.

En résumé, nous venons de comprendre :

- le principe du **cycle instruction** et du **séquencement** d'un programme,
- le rôle du **PC** qui **pointe toujours sur la prochaine instruction à exécuter**,
- les trois unités (**FETCH, DECOD** et **ARITHMETIC/LOGIC**) qui travaillent en parallèle.

Voyons maintenant la suite.

III.3. LE STATUS REGISTER

Le **REGISTRE D'ETAT** (noté **SR** comme **STATUS REGISTER**) a un peu le même rôle que celui du même nom que l'on a défini pour les circuits périphériques :

donner, à un instant donné (c'est-à-dire après chaque exécution d'instruction) une image de l'état du microprocesseur à cet instant.

Il faut savoir en effet que certaines instructions peuvent avoir des informations à transmettre à l'instruction suivante (et cela sera fait via le **SR**).

Donnons quelques exemples pour bien le comprendre et voir ainsi la nécessité pour le microprocesseur d'avoir un registre d'état :

- quand on effectue l'**addition** de 2 octets (respectivement de 2 mots), le résultat ne tient pas sur 1 octet (respectivement 1 mot) mais peut faire apparaître un 9^{ème} bit (respectivement un 17^{ème} bit), qu'on appelle le **BIT DE RETENUE** (ou **CARRY BIT**) et noté souvent **C**.
En conséquence, si on veut que le résultat de l'addition soit juste, il faut prendre en compte ce bit de **CARRY**. Et la seule façon de le signaler sera de positionner, dans le microprocesseur (plus exactement dans son **STATUS REGISTER** un bit particulier à **1**, s'il y a eu retenue et à **0** s'il n'y a pas eu de retenue suite à l'addition précédente),

- si on veut **comparer** deux opérandes, la méthode utilisée par le microprocesseur est d'en faire la différence. En fonction du signe de cette différence (et éventuellement de sa nullité si ces 2 opérandes sont égaux), le microprocesseur positionnera à **1** dans le **SR** deux bits particuliers :

- le **BIT ZERO** (ou **ZERO BIT**) noté souvent **Z** si cette différence est **nulle** (et donc si les opérandes sont égaux). Ainsi, si ce bit **Z** est à **1** après une comparaison, cela signifie que les 2 opérandes sont égaux. Et s'il est à **0**, que les opérandes sont différents.

- le **BIT NEGATIF** (ou **NEGATIF BIT**) noté souvent **N** si cette différence est **négative**. Ainsi, si ce bit **N** est à **1**, cela permettra de savoir lequel des 2 opérandes est plus grand que l'autre. Et s'il est à **0**, ce sera l'inverse.

Détaillons davantage l'utilité du bit **Z** sur un exemple concret.

Pour bien comprendre cet exemple, il ne faut pas oublier la principale caractéristique de la logique dite "**programmée**" par opposition à la logique dite "**câblée**" :

pouvoir, sur le même HARD, plus ou moins standardisé, faire tourner des SOFTS de natures différentes pour s'adapter à de multiples applications et donc à de multiples environnements.

Or l'environnement électrique (ou électronique) d'une carte évolue au cours du temps. C'est ce que l'on va détailler sur l'exemple (très (trop ???) simple) suivant :

soit un système **d'ouverture automatique** de porte (quand une personne se présente devant).

Pour cela, il existe un **capteur de présence**, situé au-dessus de la porte qui va fournir sur sa sortie par exemple un niveau haut (5V) si une personne est détectée devant la porte, et un niveau bas (0V) sinon. (niveau logique de présence = niveau haut = 1).

Au niveau **algorithmique**, il faudra coder par exemple l'**organigramme logique** de la figure III.3.

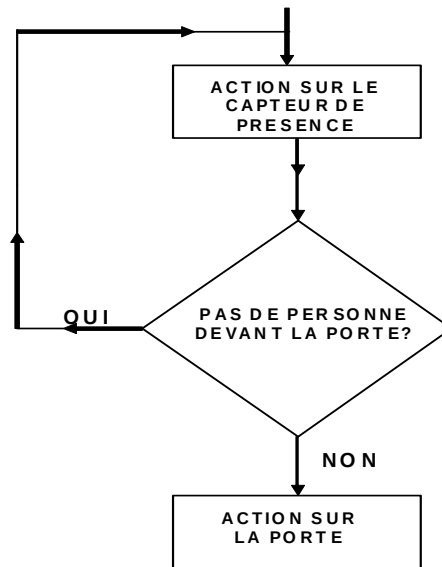


Figure III.3.

Il se lit **du haut vers le bas**, en suivant les flèches de parcours.

Un **losange** représente un **test logique** avec **2 sorties** :

- une sortie **OUI** vers la gauche (ou la droite) si la condition est **vraie**,
- et une sortie **NON** vers le bas (la suite du traitement) si la condition est **fausse**.

Un **rectangle** représente un **traitement** (une **action**, une **opération** ...) à faire.

Sa compréhension est assez intuitive :

le programme attend qu'une personne se présente devant la porte pour l'ouvrir.

Au niveau **électronique**, ce même organigramme garde sa structure générale mais les indications fournies sont plus proches du hardware (figure III.4. ci-dessous).

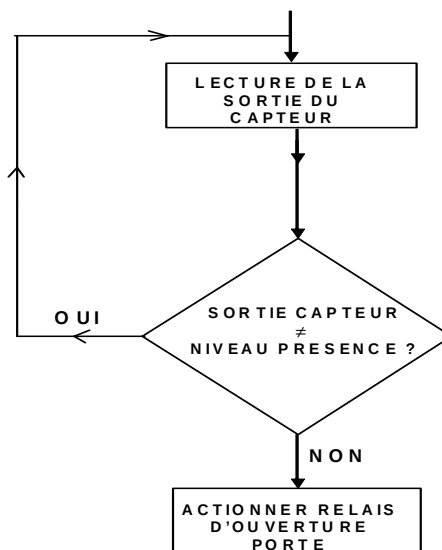


Figure III.4.

Ainsi, le traitement à effectuer par le programme codant ce petit **organigramme ne devra pas être le même** en fonction de la présence ou pas d'une personne devant la porte.

Dans un cas il attend en faisant ce qu'on appelle une boucle de **SCRUTATION (POOLING) conditionnelle**, dans l'autre cas il doit ouvrir la porte.

L'instruction à exécuter au niveau du microprocesseur pour faire le test de présence sera dans un premier temps une **comparaison** (là où on aurait utilisé un **comparateur en logique câblée**) entre la sortie du capteur et le niveau correspondant à la présence d'une personne (ici on a vu le niveau haut).

Le rôle de cette instruction de comparaison sera justement de positionner le bit **Z** du **SR** à **1** s'il y a égalité entre la sortie du capteur et le niveau logique haut de présence ou de le positionner à **0** si la sortie du capteur est au niveau bas.

Ensuite, après cette comparaison, l'**instruction suivante**, en fonction de la valeur du bit **Z**, consistera :

- soit à **refaire une comparaison** : sortie **OUI** (si **Z = 0**),

- soit à **ouvrir la porte** : sortie **NON** (si **Z = 1**).

Cette instruction est en fait **une rupture de séquence** (si $Z = 0$, au lieu de passer séquentiellement à l'instruction suivante qui consiste à actionner le relais d'ouverture de porte, on effectue à nouveau une comparaison, et ceci tant que $Z=0$. Par contre dès que Z vaudra 1, on passera à l'instruction suivante pour actionner le relais d'ouverture). On appelle cela un **saut conditionnel**.

Remarque :

En langage C, cet organigramme pourrait être codé par les 2 lignes suivantes :

```
while ( sortie_capteur != niveau_presence ) ;      * attente d'une personne  
  
Ouvrir( ) ;                                       * appel de la fonction  
                                                * d'ouverture porte
```

Mais on voit bien que quand on programme en langage évolué, on est très, très, très loin de se poser la question de l'existence dans le microprocesseur d'un registre d'état !!!

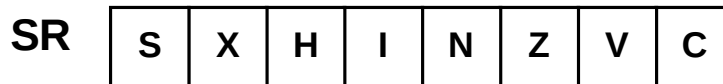
Au final, le registre d'état joue un peu le rôle de « ciment » entre les différentes instructions. Il permet de passer une « **image** » (de quelques bits) de l'état du microprocesseur, une espèce de « **nano-photographie** » de cet état qui sera utilisée par l'instruction suivante.

Notons que certaines instructions ne modifient pas ce registre d'état. D'autres vont positionner à **1** ou à **0** un ou plusieurs bits de ce registre d'état. Pour cela, il faudra consulter à chaque fois la documentation des instructions.

Ainsi, le registre d'état apparaît comme un ensemble d'indicateurs (qu'on appelle aussi des BITS DE FLAG (drapeaux) ou plus succinctement des FLAGS).

Chaque microprocesseur / microcontrôleur aura sa propre façon de les coder dans le **SR**.

Donnons par exemple sur la figure III.5. pour le **HC11** la place de chacun des **FLAGS** dans le **SR**.



Registre d'état SR du HC11

Figure III.5.

On ne détaillera pas les bits **S**, **X**, **H** et **I** du quartet fort de **SR**.

Ils sont utilisés pour des modes de fonctionnement particulier du HC11, qui n'ont pas d'intérêt ici, dans cette présentation générale.

Passons à ceux, du quartet faible.

On a vu le rôle de **C** (**CARRY BIT**), de **Z** (**ZERO BIT**) et de **N** (**NEGATIF BIT**).

Pour comprendre le rôle du **FLAG V** (comme **OVERFLOW** ou **DEPASSEMENT DE CAPACITE**), il faut faire quelques rappels sur l'arithmétique entière signée.

En codage binaire, il y a essentiellement deux types de codage pour les entiers :

- soit en binaire **non signé** (le **unsigned** du langage C),
- soit en binaire **signé** (le **signed** du langage C).

En binaire **non signé** (binaire **simple** ou **naturel**), chaque bit représente son poids en base 2 :

- sur **1 octet** on peut représenter les entiers de : **0 à $2^8 - 1$** soit de **0 à 255**,
- sur **2 octets** on peut représenter les entiers de : **0 à $2^{16} - 1$** soit de **0 à 65 535**,
- sur **4 octets** on peut représenter les entiers de : **0 à $2^{32} - 1$** soit de **0 à 4 294 967 295**.

En binaire **signé**, le bit de poids fort (le **MSB**) représente le bit de signe, noté **N** avec la convention suivante :

- si **N = 0** entier **positif ou nul**,
- si **N = 1** entier **négatif**.

Remarques :

1) Pour trouver le codage d'un entier négatif, on commence par coder sa valeur absolue en binaire non signé, puis on inverse tous ses bits et enfin on rajoute 1.

Exemple : codage de -5 sur un octet en binaire signé

Valeur absolue 5	0000 0101
Inversion des bits	1111 1010
Ajout de 1	+ 0000 0001
Soit au final	1111 1011

Et donc -5 sera codé \$FB (en signé).

2) Pour trouver la valeur d'un entier négatif codé en binaire signé, on fait la même chose.

Exemple : entier négatif codé par \$81 en signé

Valeur en binaire signé	1000 0001
Inversion des bits	0111 1110
Ajout de 1	+ 0000 0001
Soit au final	0111 1111 = \$7F

soit 127, valeur absolue de l'entier négatif
Et donc \$81 représente -127 (en décimal)

3) Si le bit de signe est à 0, le codage signé est le même que le non signé.

4) Il est intéressant de se rappeler que le nombre binaire constitué que de 1 soit :

\$FF sur 1 octet

ou

\$FFFF sur 2 octets

ou

\$FFFFFFFF sur 4 octets

ou ...

représente le codage de -1 en binaire signé (sur 1, 2, 4 ... octets)

Ainsi, lorsque l'on travaille en binaire signé :

- sur **1 octet** on peut représenter les entiers de : -2^7 à $2^7 - 1$ soit de -128 à +127,
- sur **2 octets** on peut représenter les entiers de : -2^{15} à $2^{15} - 1$ soit de -32 768 à 32 767
- sur **4 octets** on peut représenter les entiers de : -2^{31} à $2^{31} - 1$ (à calculer ...)

Par contre, si on fait l'addition de 2 entiers négatifs de forte valeur absolue le bit de signe peut disparaître (et passer de 1 à 0), entraînant un résultat qui risque d'être erroné.

Voyons cela sur un exemple avec deux entiers codés sur un octet :

si on fait $(-123) + (-60)$, le résultat devrait donner -183 .

Or : -123 sera codé : 1000 0101 en binaire signé
 -60 sera codé : 1100 0100 en binaire signé
 leur somme donnera 0100 1001 (et $C=1$) et donc ne correspond pas un nombre négatif car $N=0$.

Le résultat est faux, le bit de signe a disparu.

En fait, ce sera toujours le cas chaque fois que le résultat réel ne sera pas compris entre -128 et $+127$ (qui sont les limites de codage sur un octet en binaire signé).

Et bien le bit **V** (comme **OVERFLOW** ou **DEPASSEMENT DE CAPACITE**) est là pour signaler cette anomalie. Il joue, on le voit, un peu le même rôle que joue la carry **C** avec les entiers non signés (avec le bit **C**, on dit qu'il y a eu débordement).

En fait, si on y regarde de plus près, le microprocesseur ne sait pas (et ne peut pas savoir) s'il travaille en arithmétique signée ou en non signée. Et du coup, il fait comme s'il travaillait dans les deux cas de codage : ce sera au programmeur (ou au compilateur dans le cas de l'utilisation d'un langage évolué via les déclarations signed ou unsigned du langage C par exemple) de faire très attention et d'utiliser les instructions adéquates.

Vous n'êtes pas convaincus ? Et, bien essayez de répondre à la question suivante :

de ces 2 octets \$7F et \$80 lequel est le plus grand ?

Réponse : impossible de le savoir tel quel :

car en non signé \$80 ($= 8 \times 16 = 128$ en décimal) est plus grand que \$7F ($= 7 \times 16 + 15 = 127$ en décimal).

Mais en signé c'est l'inverse.

(car \$80 = %1000 0000 est un entier négatif alors que \$7F = %0111 1111 est un entier positif)

En pratique, il y aura deux types d'instruction de test : une à utiliser si on travaille avec des entiers non signés et une autre, différente, si on travaille avec des entiers signés.

On apprendra en manipulant en TP sur le HC11 comment bien s'y retrouver avec tous ces flags.

III.4. LE STACK POINTER

C'est certainement une des parties les plus délicates et les plus difficiles de ce cours.

Pour bien comprendre l'utilité de la **PILE (STACK)** et du **POINTEUR DE PILE (STACK POINTER)**, il est préférable d'avoir un peu programmé en langage de bas niveau. Et c'est justement un des objectifs de ce cours : donner des notions sur la programmation de bas niveau ...

Donc tout cela semble un peu « se mordre la queue » !!!

Pour s'en sortir, on essaiera dans cette présentation, de faire le lien avec un langage évolué comme le C. Mais comme on l'a vu dans l'exemple avec le registre **SR**, la programmation en langage évolué (qui comme son nom l'indique se situe à un niveau bien au-dessus et loin des aspects purement électroniques et d'architecture de système) masque beaucoup de problèmes liés à la structure du hard (ce qui est en fait normal car c'est un des buts recherché avec les langages évolués).

Conclusion provisoire : des éléments supplémentaires sur la pile seront fournis par la suite, que vous comprendrez mieux après avoir un peu fait de la programmation sur le HC11.

Que peut-on cependant dire ici pour introduire ces notions ?

Si on réfléchit un peu plus à la programmation et au traitement de l'information, on peut imaginer que certaines tâches puissent être exécutées plusieurs fois et à des moments différents par un même programme.

Par exemple :

- la tâche de recevoir un nouveau caractère arrivant sur une liaison (et qui sera répétée à chaque nouveau caractère reçu pour le ranger dans un buffer de réception),
- la tâche d'afficher un point (pixel) en (X,Y) sur un écran graphique (et qui sera répétée plusieurs fois (en changeant bien sûr les coordonnées) pour tracer une droite, ou un cercle),
- la tâche de faire une conversion entre deux types de codage (hexadécimal vers ASCII pour afficher un chiffre sur un afficheur en mode texte et qui sera répétée plusieurs fois pour afficher un nombre (plusieurs chiffres) ...

On le voit les exemples sont nombreux et rappellent la notion de **FONCTION** définie en langage C : on définit puis on écrit une fonction particulière et on l'appelle depuis le **PROGRAMME PRINCIPAL** (le **MAIN**) chaque fois qu'on en a besoin.

Plus généralement, cet aspect sous-entend la notion de **SOUS-PROGRAMME**. On parlera aussi de **MODULE** quand on programmera en assembleur.

Qu'est-ce c'est ?

C'est en fait un bout de programme, avec un début et une fin, conçu pour une tâche (un travail) précise et qui sera appelé par le programme principal chaque fois que cette tâche devra être effectuée. Il est clair qu'à la fin de ce sous-programme, " la main " (c'est-à-dire le contrôle) doit être rendu au programme principal pour continuer normalement la suite du traitement.

Comme en langage C : quand on appelle une fonction, elle s'exécute sur le microprocesseur et quand on arrive à son accolade fermante (**}**) ou quand elle effectue un **return** (valeur_retournée), on **revient** dans le main à l'endroit **situé juste après l'appel de la fonction** pour continuer la suite du programme principal.

Cette possibilité de faire des sous-programmes (des fonctions en C) est très intéressante pour de multiples raisons :

- **gain de mémoire** : le code de la fonction n'existe qu'en un seul exemplaire en mémoire, alors qu'il peut être exécuté autant de fois que nécessaire,
- on **découpe** un programme complet et complexe en **bouts plus simples**, ce qui **facilite sa conception** et améliore sa **lisibilité** et son **évolutivité**,
- chaque fonction à utiliser peut être testée **individuellement** (et parfois **indépendamment**) du programme principal : comme elle est en général plus simple que le main, cela **facilite les tests** et la **mise au point**,
- l'assemblage de fonctions toutes testées individuellement (et à **150%**) transforme la conception du programme principal en « **un jeu de Léo** »,
- certaines fonctions (très **générales** et **génériques**) pourront avantageusement être **réutilisables** dans un autre contexte. Il n'y a qu'à voir la richesse des fonctions disponibles dans les **bibliothèques** du C pour s'en convaincre. De plus, une fonction issue d'une bibliothèque a été en principe validée complètement : donc si votre programme qui l'utilise ne marche pas, l'erreur vient sûrement de vous (par exemple à cause d'une mauvaise interprétation de la documentation de cette fonction ...).

Bref beaucoup d'avantages.

Comment cela peut-il marcher au niveau du microprocesseur ?

Il est à priori facile avec notre modèle simplifié de microprocesseur de donner la main à un sous-programme.

Exercice :

Quel registre est concerné par cette opération ?

Mais un problème se pose pour le **retour** au programme principal (on dit aussi au programme **APPELANT** (**CALLING PROGRAM**) par opposition au sous-programme dénommé alors programme **APPELE** (**CALLED PROGRAM**).

En effet n'oublions pas que pour s'exécuter, le sous-programme utilise les ressources du microprocesseur, avec notamment le **PC** qui a été modifié pour pointer sur la zone d'emplacement mémoire du sous-programme, différente de celle du programme principal.

Un peu comme quand on quitte un premier livre (le programme appelant) en le fermant pour en lire un autre (le sous-programme appelé) et qu'on revient au premier livre ... on a perdu la page ... sauf si on a pris la précaution, avant de quitter le premier livre, de repérer la page où on en était.

De même, le microprocesseur aura perdu l'emplacement de la zone correspondant au programme appelant (valeur du **PC** avant l'appel du sous-programme) ... sauf si comme vous « il a marqué la page » en prenant la précaution de sauvegarder la valeur de retour.

On dit qu'il doit sauvegarder l'**ADRESSE DE RETOUR** ou **RETURN ADDRESS** (celle de la case mémoire qui suit l'appel du sous-programme), valeur qu'il récupérera lorsque le sous-programme rendra la main au programme principal, garantissant, comme en langage C, un retour à l'endroit exact situé juste après l'appel de la fonction pour continuer la suite du programme principal.

Ici deux solutions simples sont envisageables :

- soit l'existence d'un registre interne au microprocesseur dans lequel il vient sauvegarder cette **ADRESSE DE RETOUR** (et qu'il récupérera dans son PC à la fin du sous-programme),
- soit faire cette sauvegarde à l'extérieur du microprocesseur, directement en mémoire.

La première solution, qui semble plus simple et plus rapide (puisqu'elle évite un **CYCLE BUS** dans la mémoire externe) a l'inconvénient de ne pouvoir être utilisée qu'une seule fois : on ne peut plus appeler un deuxième sous-programme à partir d'un premier sous-programme (appels imbriqués). Ou alors il faut un deuxième registre à l'intérieur, puis au troisième en cas d'une autre imbrication et ainsi de suite.

La seconde, même si elle est plus lente, semble plus satisfaisante car :

- après tout, il y a de la mémoire à l'extérieur, alors autant l'utiliser,
- la taille de la mémoire extérieure permettra facilement les appels imbriqués multiples.

Par contre, il ne faut pas mélanger la mémoire " normale " (celle pour le programme ou pour les données du programme) avec la zone de mémoire dédiée pour cette sauvegarde un peu particulière.

Cette zone de mémoire dédié est appelée la **PILE** (**STACK**).

Exercice :

De quel type de mémoire est constituée une pile ? Pourquoi ?

Et c'est le rôle du registre **POINTEUR DE PILE (STACK POINTER, SP)** d'indiquer où se trouve cette **PILE** en mémoire.

Comme son nom l'indique, il contient (on dit qu'il **POINTE** vers) une adresse mémoire correspondant à l'emplacement mémoire où le microprocesseur sauvegardera (entre autre) l'**ADRESSE DE RETOUR** (en cas d'appel d'un sous-programme).

Il pointe donc sur le **SOMMET** de la **PILE (STACK TOP)** c'est-à-dire sur la **première case mémoire disponible sur le haut de la pile**.

Exercice :

Quand faut-il initialiser le registre **STACK POINTER** ? Pourquoi ?

Précisons quand même que dans le cas des sous-programmes, il existe des instructions particulières qui permettent de simplifier la gestion de la pile et de la rendre (presque et peut-être parfois **trop**) **TRANSPARENTE** pour le programmeur.

Par exemple :

- l'instruction d'appel d'un sous-programme **sauvegarde automatiquement** la valeur du **PC** (c'est-à-dire comme on l'a vu l'**ADRESSE DE RETOUR**) dans la pile, (on dit aussi **EMPILE** le **PC** ou **PUSH** le **PC**), **décrémente** ensuite tout aussi **automatiquement** le **SP** pour que ce **SP pointe toujours** sur le **SOMMET** de la pile (on voit donc que la pile **grandit vers les adresses basses** car **SP** est **décrémenté** à chaque **empilement**), puis enfin **charge** dans le **PC**, encore une fois **automatiquement l'adresse de la première instruction du sous-programme**.
- l'instruction de sortie du sous-programme **replace** dans le **PC** cette valeur sauvegardée (on dit aussi **DEPILE l'ADRESSE DE RETOUR**, ou **POP l'ADRESSE DE RETOUR**) pour assurer la suite du programme principal au bon endroit et **incrémente** ensuite tout aussi **automatiquement** le **SP** pour que ce **SP pointe toujours** sur le **SOMMET** de la pile.

Exercice :

Quelle adresse de la RAM a t'on intérêt à choisir pour initialiser le **SP** ? Pourquoi ?

Voyons tout ce mécanisme dans le cas d'un exemple simple illustré sur la figure III.6. :

- cf. ① : **programme principal** implanté entre les adresses \$3000 et \$4FFF,
- cf. ② : **sous-programme** implanté à la suite entre les adresses \$5000 et \$5FFF,
- cf. ③ : **sommet de la pile** en valeur initiale à l'adresse \$7FFF.

On suppose de plus (cf. ④) que **l'appel du sous-programme** se fait par l'instruction située à l'adresse \$4000 du programme principal (en fait instruction d'appel est codée sur 2 octets en \$4000 et \$4001)

On suppose (cf. ⑤) qu'au départ le **PC** contient \$3000 (adresse du début du programme principal) et que le **SP** sera initialisé (par la première instruction du programme) avec \$7FFF (adresse la plus haute de la RAM qui devient ainsi le **sommet de la pile** cf. ⑥).

Ce qui est résumé sur la configuration mémoire suivante (figure III.6.) :

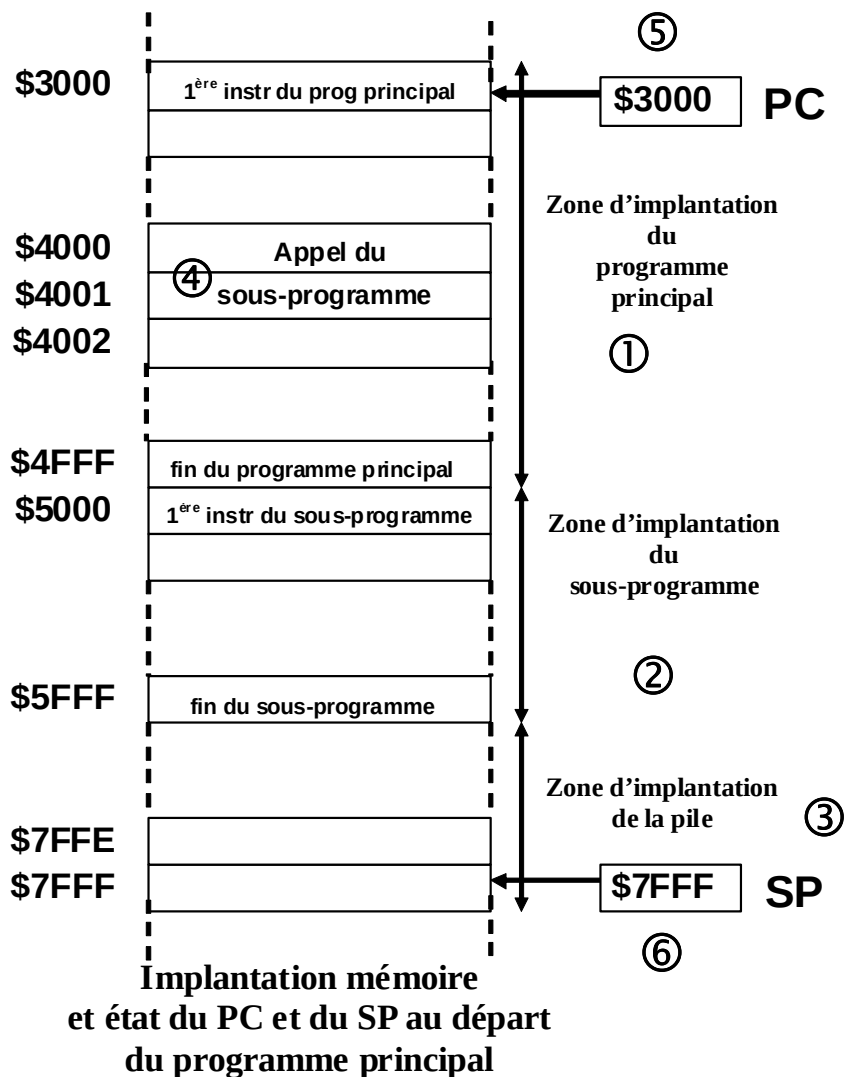


Figure III.6.

Et le programme principal commence à s'exécuter

Lorsque le **PC** contiendra l'adresse **\$4000**, il va exécuter l'appel du sous-programme.
(on a supposé ici que ce code tient sur 2 octets : en **\$4000** et **\$4001**)

On verra qu'en langage assembleur HC11 c'est le code **JSR** (**J**ump to **S**ub**R**outine) en adressage **absolu** ou **BSR** (**B**ranch to **S**ub**R**outine) en adressage **relatif** suivi du nom du sous-programme, qui n'est rien d'autre que l'adresse de la première instruction du sous-programme (ici **\$5000** mais en pratique, on utilisera un **label**, comme pour les sauts).

Il se passe alors, et dans cet ordre, les **automatismes** suivants (cf figure III.7.) :

- ① sauvegarde du **PC** dans la pile (n'oubliez pas que le **PC** contient alors **\$4002** car **il pointe toujours sur l'instruction suivante** (**\$4002** est ainsi l'**adresse de retour**)
- ② le **SP** est décrémenté (ici de 2 car ici l'**adresse de retour** tient sur 2 octets et chaque case mémoire ne peut stocker que 1 octet),
- ③ la valeur **\$5000** (**adresse du début du sous-programme**) est placée dans le **PC**.

Le sous-programme peut commencer (comme indiqué sur la figure III.7.) à s'exécuter ...

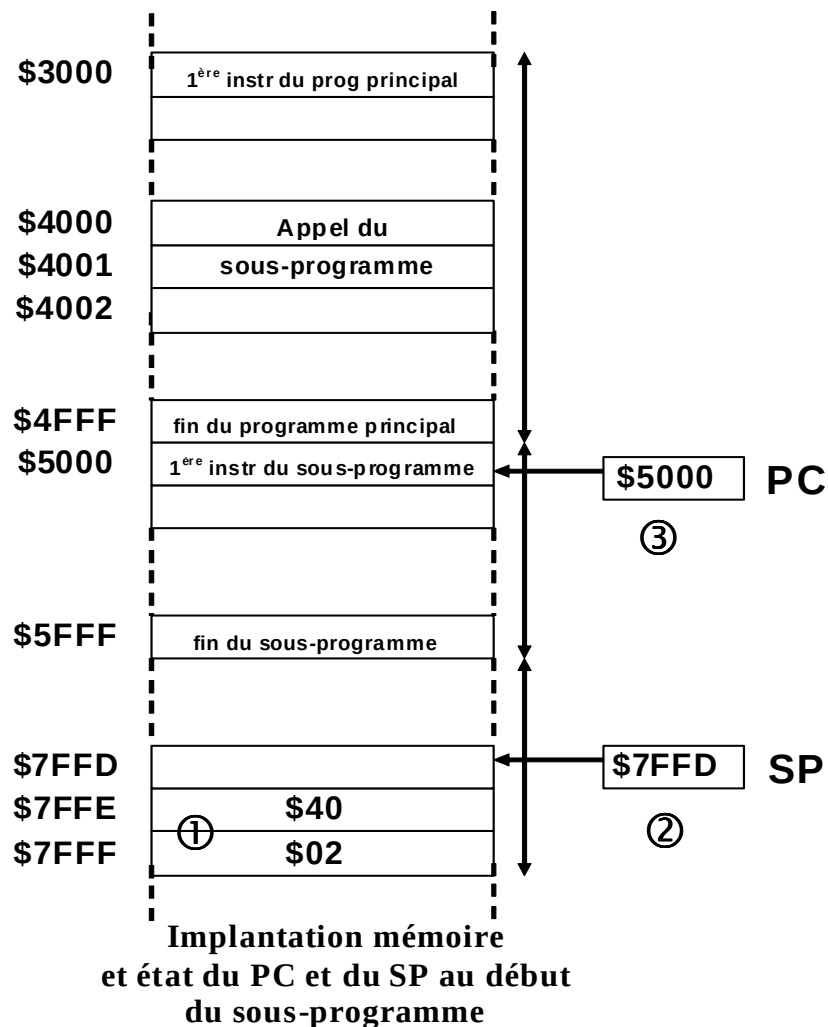


Figure III.7.

Lorsque le **PC** contiendra l'adresse **\$5FFF**, il va exécuter la dernière instruction du sous-programme qui lui permet de revenir au programme principal.

(on verra qu'en langage assembleur HC11 c'est le code **RTS** (**ReT**urn from **S**ubroutine)).

Il se passe alors, et dans cet ordre, les **automatismes** suivants (cf figure III.8.) :

- ① chargement du **sommet de la pile** (et donc de l'**adresse de retour \$4002**) dans le **PC**)
- ② le **SP** est incrémenté (ici de 2 car l'adresse de retour tenait sur 2 octets).

La suite du programme principal (juste après l'appel du sous-programme) peut continuer à s'exécuter comme on le voit sur la figure III.8. :

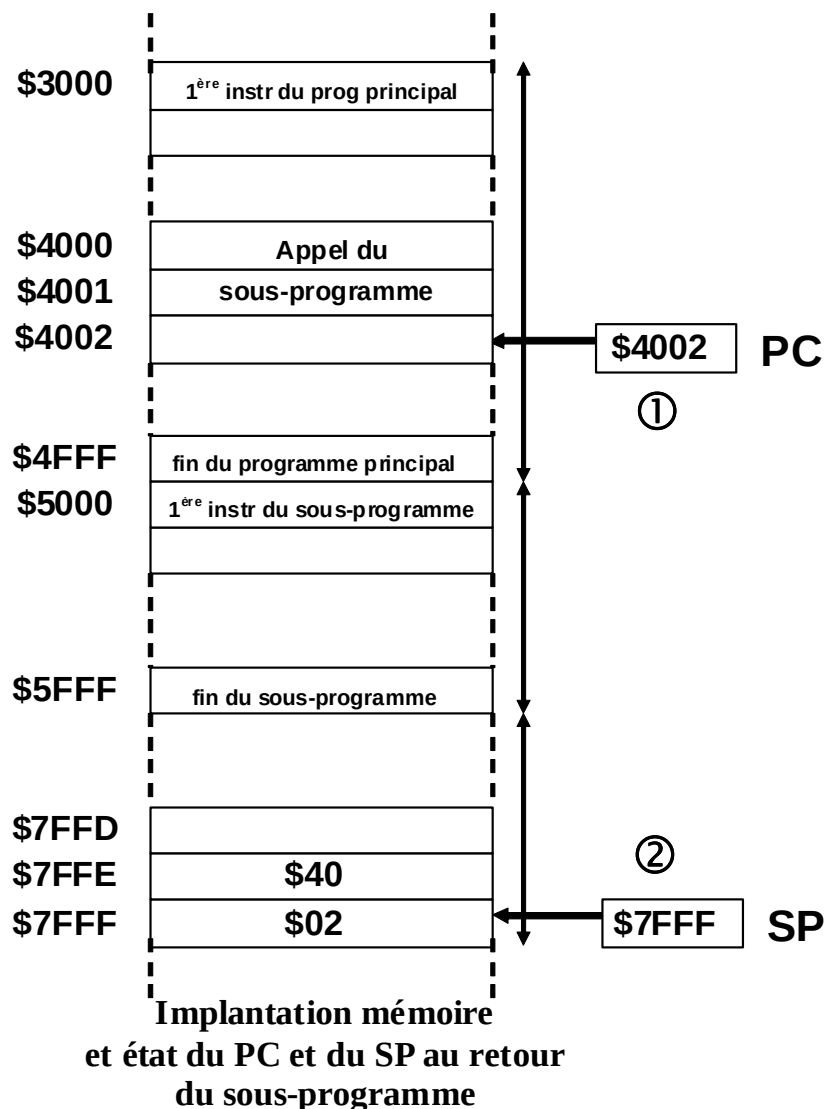


Figure III.8.

Remarque : la valeur **\$4002** est toujours présente sur la pile mais **maintenant n'a plus d'intérêt** (car le retour au programme principal a été effectué). Le **SP** pointe sur la **première case mémoire disponible de la pile** et donc si on a une autre utilisation de la pile à partir de là, cette valeur de **\$4002** sera écrasée.

Finalement, grâce à ce mécanisme, un sous-programme (ou une fonction du langage C) n'est rien de plus qu'une « **parenthèse** » (effectuant un certain travail) au sein d'un programme principal.

Ce qui était bien le but recherché.

Après son exécution, le main doit continuer la suite de son traitement un peu comme s'il ne s'était rien passé, à ceci près que le sous-programme s'est exécuté et a fait le travail qu'on lui demandait de faire. (le PC doit récupérer l'adresse de l'instruction qui suit l'appel du sous-programme et le SP doit retrouver la valeur qu'il avait au moment de l'appel du sous-programme).

La pile est une structure de type **LIFO** (**LAST IN FIRST OUT**) ou dernier (élément) rentré, premier sorti, par opposition aux structures dites **FIFO**. (**FIRST IN FIRST OUT**) ou premier (élément) rentré, premier sorti. (comme dans une **QUEUE** ou **FILE D'ATTENTE**).

Exercice :

Pourquoi cette structure pour la pile ? Essayer de la justifier en trouvant un mécanisme cohérent pour la pile et en vous aidant d'un petit schéma.

Aide : pensez à l'appel d'un deuxième sous-programme à l'intérieur d'un premier.

Le **PC** est-il le seul registre à devoir être sauvegardé dans la pile ? Pourquoi ?

De façon générale, le travail avec la pile est délicat.

Nous n'en présenterons pas ici le détail car nous avons seulement voulu montrer la nécessité d'avoir à l'intérieur de tout microprocesseur un registre type **STACK POINTER** (lié à la nécessité d'avoir une **PILE** en mémoire).

Par contre, les mauvaises manipulations avec la pile peuvent entraîner des conséquences fâcheuses de " plantage " systématique pour le logiciel (on dit dans le jargon des informaticiens que l'on a fait " exploser " sa pile), pas toujours faciles à débiter.

Vous aurez l'occasion de vous en rendre compte en Travaux Pratiques !!!

Mais, au final, la pile est indispensable pour :

- réaliser le **passage d'informations au sous-programme** (comme les **paramètres** ou les **arguments** d'une fonction en langage C, qui sont mis sur la pile avant l'appel de la fonction),
- gérer des **variables locales** à une fonction (variables **automatiques** du langage C), qui sont créées sur la pile (juste au-dessus des arguments et de l'adresse de retour, au point d'entrée dans la fonction),
- le traitement des **interruptions**,
- la **récurivité** des sous-programmes (sous-programme s'appelant lui-même),
- les **appels système**,
- les **commutations** entre tâches dans les systèmes **multitâches** et **multiutilisateurs** ...

III.5. LES AUTRES REGISTRES

Hormis le **PC**, le **SR** et le **SP**, liés au fonctionnement du microprocesseur, il y a d'autres registres internes dans le microprocesseur qui de façon générale serviront :

- de **R**egistres (de travail) pour les **D**onnées (**RD_i**) : stockage des informations et des résultats de calculs ou de traitements intermédiaires pour les données (variables ou constantes),

- de **R**egistres (de travail) pour les **A**dresses (**RA_i**) : pour calculer des adresses d'opérandes, ils stockeront des pointeurs vers les cases mémoire contenant les opérandes.

On les appelle aussi parfois des registres **d'INDEX** (**INDEX REGISTER**).

Le **nombre**, le **type** et la **taille** de ces registres dépendra de chaque microprocesseur.

Et aussi très fortement des microprocesseurs précédents dans le cas d'une **lignée** de microprocesseurs. Cette notion de famille « historique » imposant ce type de contraintes.

En effet, les microprocesseurs suivants (et plus performants) dans une lignée devront avoir un minimum de **compatibilité** (on parle de **COMPATIBILITE ASCENDANTE** ou **MONTANTE**) avec leurs « **ancêtres** » pour que les logiciels développés pour ces "ancêtres" **continuent à fonctionner** sur les " **descendants** ".

A titre d'exemple, citons deux familles parmi les plus connues :

- **Motorola** : 68 000, 68 010, 68 020, 68 030, 68 040, .Power PC ...

- **Intel** : 8086, 80 186, 80 286, 80 386, 80 486, Pentium I, II ...

Note :

La notion de **registre ACCUMULATEUR**, si fréquente dans les microprocesseurs 8 bits tend à disparaître sur les microprocesseurs de génération plus récente, au profit d'une banalisation de tous les registres généraux (qui peuvent en fait servir d'accumulateur).

Rappelons que dans le cas des microprocesseurs de 8 bits, l'accumulateur désigne un **registre spécial** qui est en quelque sorte une des entrées (d'opérande) et la sortie de l'U.A.L.

Toutes les opérations élémentaires sont ainsi effectuées via cet accumulateur et fournissent leur résultat dans cet accumulateur (qui devient une sorte de « passage obligé » pour toutes les opérations de traitement dans l'U.A.L.).

Le tableau ci-dessous résume un peu l'état de nos connaissances.

LES UNITES INTERNES	REGISTRES LIES AU FONCTIONNEMENT DU MICROPROCESSEUR	REGISTRES DE TRAVAIL
FETCH UNIT	PC : pointe sur la prochaine instruction	DATA REGISTERS : RD_i ADDRESS REGISTERS : RA_i
DECODE UNIT	SR : registre des flags	
ARITHMETIC/LOGIC UNIT	SP : pointe sur le sommet de la pile	

III.6. PROPOSITION D'ARCHITECTURE INTERNE (SIMPLIFIEE)

Nous sommes maintenant capables de présenter un **modèle d'architecture interne** de microprocesseur mettant en évidence les points précédemment abordés (cf figure III.9.).

Encore une fois, ce modèle ne correspond pas à un composant réel, mais on y trouve tous les grands principes qui existent dans les microprocesseurs réels.

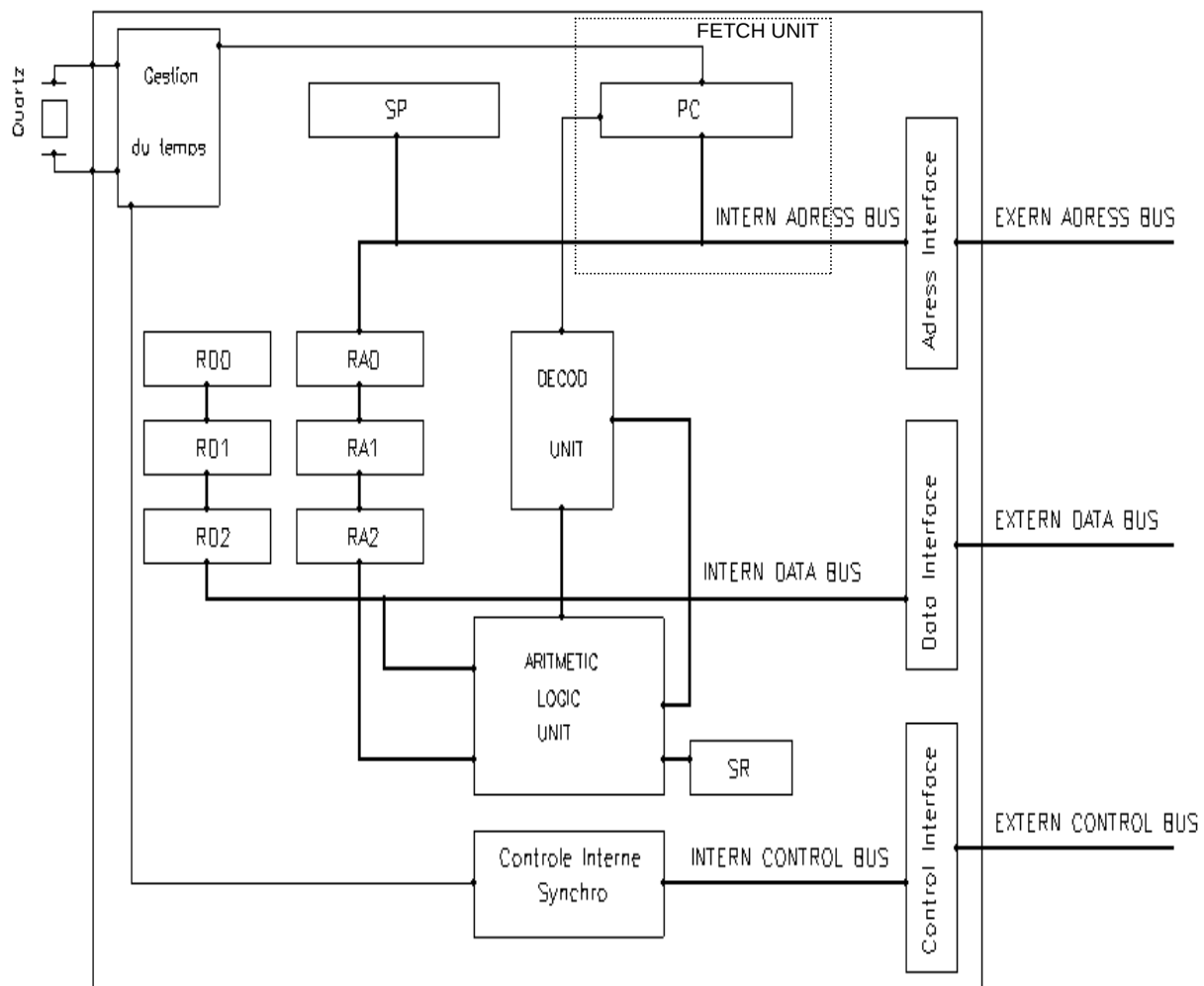


Figure III.9.

Remarque : on distingue sur ce schéma les **DIFFERENTS BUS INTERNES (ADDRESS, DATA et CONTROL)** réalisés en piste de silicium dans le composant. La largeur de gravure de ces pistes ne cesse de diminuer et fournit un indicateur de la puissance d'intégration de la technologie. Ne pas confondre ces bus internes avec les **BUS EXTERNES** constitués, rappelons-le, de pistes de cuivre sur le circuit imprimé...

III.7. SYNTHÈSE : LE MODÈLE DE PROGRAMMATION

Pour un microprocesseur donné, on définit ce qu'on appelle son :

MODÈLE DE PROGRAMMATION (PROGRAMMATION MODEL)

C'est en fait la **liste exhaustive** de ses registres internes accessibles au programmeur.

C'est en quelque sorte l'interface entre le **HARD** (le microprocesseur) et le **SOFT** (le programmeur) ou le minimum de hard que doit connaître le programmeur pour mettre en œuvre ce microprocesseur.

Dans le cas de notre microprocesseur simplifié, le **MODÈLE DE PROGRAMMATION** pourrait avoir la forme donnée sur la figure III.10. :

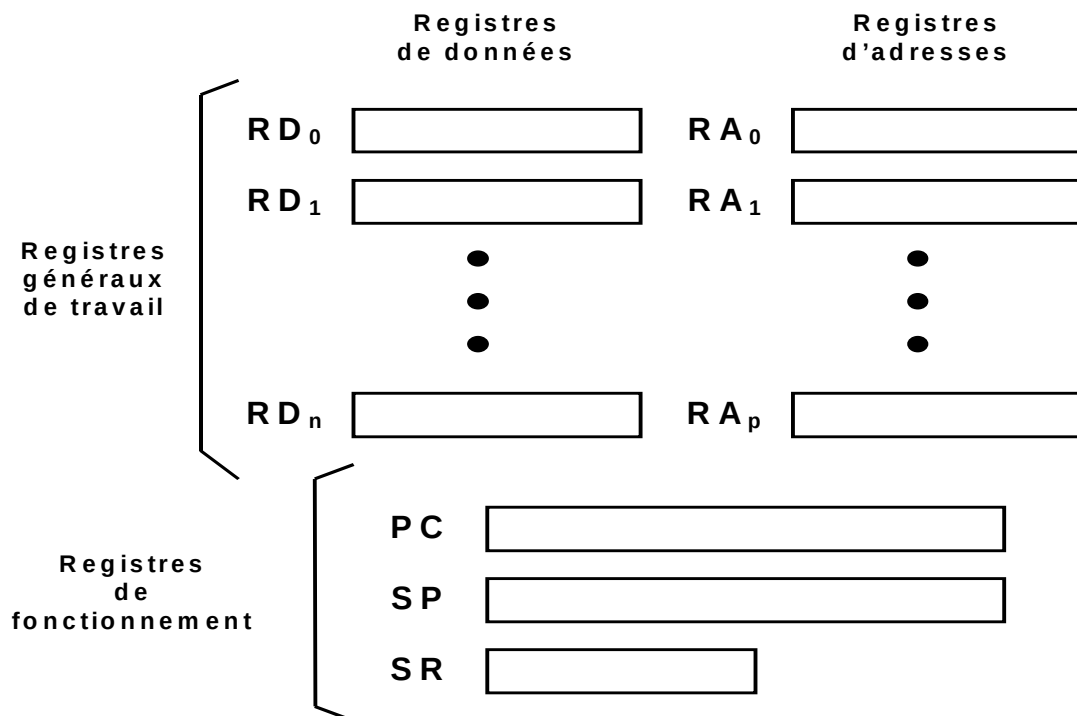


Figure III.10.

Deux remarques :

- la **taille** de ces registres internes (8 bits, 16 bits, 32 bits, 64 bits, 128 bits ...) dépendra de chaque microprocesseur,
- ces registres sont dans la CPU du microprocesseur. Contrairement à ceux des circuits périphériques et/ou aux cases mémoire (et à ce que pensent certains étudiants),

ils n'ont pas d'adresses dans le mapping du système

ils seront **reconnus** en programmation directement par leur **nom générique**.

IV. LES ASPECTS LOGICIELS

Disons le tout de suite, ils sont vastes et seront revus plus en détail dans la partie du cours sur le microcontrôleur HC11.

Ici, nous donnerons des généralités qui nous permettront de mieux comprendre l'évolution et l'historique de la programmation ainsi que des éléments qui nous aideront à mettre en évidence les liens qui existent entre le **HARDWARE** et le **SOFTWARE** (le lien entre l'électronique et sa programmation).

Une analogie : nous avons, dans les parties précédentes, présenté les « cellules » du cerveau (les neurones : « notre » hardware), nous allons voir maintenant l'information qui circule entre elles (les « influx cérébraux » : « notre » software). Fin de l'analogie.

C'est pour cela que dans cette partie, nous nous limiterons aux aspects " proches " du matériel, c'est-à-dire d'abord au **LANGAGE MACHINE** et ensuite surtout au **LANGAGE ASSEMBLEUR**.

Les notions relatives aux **LANGAGES EVOLUES** et / ou aux **SYSTEMES D'EXPLOITATION** ne seront pas (ou peu) traitées ici.

On peut cependant dire sur la programmation en général qu'elle résulte d'une volonté de construire une interface utilisateur la plus proche possible de notre langage naturel, en construisant un édifice complexe sous la forme de " **BOITES DE BOITES** ", comme dans l'image suivante :

- du sable et du ciment pour faire des briques,
- des briques pour faire des murs,
- des murs pour faire des maisons,
- des maisons pour faire des villes ...

IV.1. LE LANGAGE MACHINE

A notre stade de présentation, nous avons vu qu'une **instruction élémentaire** n'était en fait qu'une **combinaison particulière** (et **spécifique** à chaque instruction) constituée d'une séquence de **1** et de **0**.

C'est le **LANGAGE MACHINE**, langage de base des microprocesseurs, et ils ne comprennent que cela.

Les utilisateurs (pauvres humains que nous sommes) ne peuvent travailler de cette façon.

... sauf les premiers électroniciens / développeurs qui **ne disposaient que** de ce moyen de communication avec les premiers microprocesseurs.

Rendons leur grâce de ce long et fastidieux travail qui consistait à bouger des interrupteurs (ouverts, fermés) pour programmer une mémoire et à lire des L.E.D. (allumées, éteintes) pour visualiser et contrôler le résultat du travail effectué par le microprocesseur

C'est un peu grâce à eux que nous disposons d'outils plus efficaces et plus faciles à utiliser : la première « brique » étant celle du langage **ASSEMBLEUR** (ou de l'**ASSEMBLAGE**).

IV.2. UNE AMELIORATION : LE LANGAGE ASSEMBLEUR

Pour faciliter la programmation, chaque instruction machine a été représentée par un **MNEMONIQUE** (**MNEMONIC**).

Ce terme vient de " **moyen mnémotechnique** " :

chaque combinaison significative de 1 et de 0 correspondant à une instruction exécutable est transcrite par un mot de vocabulaire d'un maximum de 4 à 5 lettres rappelant le travail de base réalisée par cette instruction.

Ainsi chaque microprocesseur / microcontrôleur sera caractérisé par son **propre JEU D'INSTRUCTIONS (INSTRUCTIONS SET)** fourni dans sa documentation technique.

Ce jeu d'instructions diffère d'un microprocesseur à l'autre et c'est essentiellement une des difficultés de la programmation avec ces **MNEMONICS**. Ce n'est pas la seule.

Par exemple, et pour le **HC11** :

- pour les traitements **arithmétiques** :

ADD : pour l'**ADD**ition,
MUL : pour la **MUL**tiplication,
DIV : pour la **DIV**ision ...

- pour les traitements **logiques** :

AND : pour un **AND** logique (fonction logique ET),
OR : pour un **OR** logique (fonction logique OU),
CMP : pour une **CoMP**araison ...

Noter que l'on retrouve ici, mais cette fois-ci sous forme d'instructions, la plupart des fonctions logiques que vous connaissez en logique câblée sous la forme de circuits intégrés discrets. On comprend mieux maintenant la différence entre la logique câblée et la logique programmée ...

• pour les **transferts de données** : très fréquentes, car le microprocesseur passe la plupart de son temps à déplacer les données entre ses registres internes et la mémoire externe (cf figure IV.1.).

ST : pour un transfert du **microprocesseur vers l'extérieur** (pour **ST**ore : stocke),
 (cycle d'écriture dans la mémoire externe)

LD : pour un transfert de **l'extérieur vers le microprocesseur** (pour **LoaD**: charge),
 (cycle de lecture de la mémoire externe)

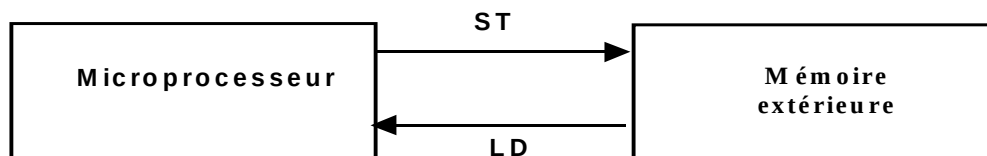


Figure IV.1.

Remarque : en général, pour la plupart des autres microprocesseurs c'est plutôt un MOVE ou un MOV (comme déplace, bouge ...)

• pour les **ruptures de séquences** :

JMP : pour un saut inconditionnel (pour **JuMP** : saute)

JSR : pour l'appel d'un sous-programme (pour **Jump to SubRoutine**)

RTS : pour la fin d'un sous-programme (pour **Return from SubRoutine**) ...

Cette liste n'est pas du tout exhaustive et, rappelons le chaque microprocesseur possédera son propre jeu d'instructions et donc ses propres mnémoniques.

En principe, comme on l'a dit au tout début de ce polycopié, indiquer au microprocesseur le type d'opération qu'il a à effectuer n'est pas suffisant, dans certains cas, pour déterminer **complètement** le travail à faire.

Par exemple :

- pour un **ADD**, il faut aussi indiquer entre quelles valeurs il doit faire l'addition,
- pour un **ST**, il faut lui préciser l'origine et la destination du transfert,
- pour un **JSR**, il faut rajouter l'adresse où se trouve la première instruction du sous programme ...

Toutes ces informations supplémentaires qui permettent de préciser complètement certaines instructions sont appelées les **OPERANDES DE L'INSTRUCTION**.
(**INSTRUCTION OPERANDS**).

L'ensemble ainsi défini et constitué par :

UN MNEMONIQUE + (éventuellement) UN (OU PLUSIEURS) OPERANDE(S),

correspond à ce que l'on appelle le **LANGAGE ASSEMBLEUR** (ou plus succinctement l'**ASSEMBLEUR**).

Programmer en **assembleur** consiste à utiliser ce langage.

Il ne faut pas oublier que le microprocesseur ne comprend pas directement ces mnémoniques.

On utilisera donc un logiciel pour transformer ces mnémoniques (et leurs opérandes) dans la suite de 1 et de 0 correspondant à ces instructions (seul langage compréhensible par le microprocesseur).

Ce logiciel s'appelle aussi par extension l'**ASSEMBLEUR**. C'est en quelque sorte un **COMPILATEUR** entre le code écrit en assembleur et le code correspondant en langage machine.

Il y a essentiellement 2 intérêts à faire un peu d'assembleur :

- un intérêt **pédagogique** qui permet de mieux comprendre le fonctionnement d'un microprocesseur et de faire plus facilement le lien entre les aspects purement matériels (problèmes électroniques d'interface, fonctionnement sous interruption, temps réel ...) et les aspects liés au déroulement d'un programme. Quand on programme en langage évolué, on est souvent très loin de ce genre de problématique ... Autrement, il ne servirait à rien de faire un cours sur les microprocesseurs et les architectures de systèmes si derrière on ne le mettait pas en pratique avec un langage de « bas niveau » comme l'assembleur qui permet de visualiser facilement et de comprendre beaucoup de notions purement électroniques.

- dans certains cas **très pointus** où il faut un traitement **très rapide**, l'assembleur peut permettre de satisfaire ces contraintes temporelles (drivers de bas niveau pour un écran graphique, pilotes de périphériques, gestion des interruptions temps réel , communications rapides ...) en réalisant un code **compact** et très **concis**. Quand on programme en langage évolué, si on n'a pas un compilateur performant, on peut être limité par ses aspects de rapidité de traitement.

Mais il existe des inconvénients à la programmation en assembleur, les principaux étant :

- **temps de développement très longs** : écriture, tests et mise au point,
- langage **peu structuré** : **absence d'instructions de contrôle de haut niveau** (comme la boucle **for**, le test conditionnel **if ... else ...**, la boucle conditionnelle **while** ou **do... while...** ou la sélection par **switch**) qui permettent de structurer tout programme,
- langage **peu lisible** : en assembleur, si on ne prend pas un minimum de précautions, notamment si on ne s'efforce pas de pratiquer une programmation dite **modulaire** (on verra dans la deuxième partie de ce cours et aussi en Travaux Pratiques comment), le programme devient vite **illisible**, **incompréhensible** et **très difficile à mettre au point**,
- **non portabilité** des sources : un programme écrit en assembleur, ne pourra fonctionner que sur un type de microprocesseur / microcontrôleur donné. Si on le change, il faudra tout réécrire, car les mnémoniques seront différentes. En langage C, il suffira de changer le compilateur, et en principe le source du C restera inchangé (avec cependant toujours quelques petites modifications à faire ...)
- quasiment **impossible** à utiliser sur des programmes **trop importants** ...

Nous aurons l'occasion de nous familiariser avec le langage et les mnémoniques du HC11 et de réaliser des petits programmes en Travaux Pratiques.

IV.3. LE DEVELOPPEMENT D'UNE APPLICATION

On suppose ici que le **HARD** est au point (testé et opérationnel). Ce sera la carte électronique comportant son microprocesseur / microcontrôleur, la mémoire (RAM et ROM) et toutes les interfaces (circuits périphériques) nécessaires à votre application.

Remarque sur cette supposition :

Cette condition n'est pas toujours remplie : quand on développe une nouvelle carte, ou une nouvelle interface ... bref quand on fait une nouvelle application électronique, il y aura toujours des erreurs purement **électroniques** (problèmes de câblage, erreurs de schémas, composants défectueux ..). Et dans ce cas, il n'est pas évident de faire la part entre ce qui relève des problèmes de hard et ce qui relève des problèmes de soft. Une règle dans ce cas : séparer les problèmes. Autrement dit, quand on teste le hard (pour le valider) utiliser des softs très, très simples qui sont sensés fonctionner à 150 %. Puis quand le hard est au point, passer aux softs de votre application.

Développer une nouvelle application comprend toutes les phases comprises entre le cahier des charges initial et les tests " in situ " du (ou des) programmes réalisés.

La méthode présentée ici est assez générale pour être indépendante du microprocesseur / microcontrôleur cible utilisé, du langage de programmation choisi et des **OUTILS DE DEVELOPPEMENT (DEVELOPMENT TOOLS)** à disposition (ensemble des moyens matériels et logiciels à mettre œuvre pour la conception, les tests et la mise au point de l'application).

Bref elle s'applique quasiment à la plupart des environnements.

On aura d'ailleurs l'occasion de la mettre en œuvre sur le HC11 en Travaux Pratiques.

Cette méthode comprend plusieurs phases, que l'on va maintenant détailler.

IV.3.1. ANALYSE

Après lecture du cahier des charges (qu'on s'assure d'avoir bien compris), on commencera toujours par une phase d'analyse qui permettra de poser rationnellement le problème à résoudre :

- organigrammes de traitement,
- découpage en « **bouts** » plus simples (fonctions, sous-programmes, modules ...) indépendants et donc testables individuellement,
- utilisation de pseudo-code ...

Cette phase peut se faire en plusieurs passes, par approches successives. Elle est importante et une mauvaise analyse peut conduire à des logiciels non maintenables ou à une « inflation » des temps de développement. C'est la phase où on montre « **toute sa matière grise** ».

Si ce n'est pas imposé dans le cahier des charges, cette phase peut aussi poser le problème du choix du microprocesseur, du langage de programmation et des outils de développement.

IV.3.2. ECRITURE DU SOURCE

Ensuite on utilisera son traitement de textes favori pour écrire le code **SOURCE** dans le langage choisi. (et on mettra le **maximum** de commentaires pour faciliter sa compréhension). Cette phase sera effectuée la plupart du temps avec un ordinateur différent de votre carte applicative, souvent un simple ordinateur personnel comportant tous les logiciels nécessaires au développement. (votre carte applicative est à **l'extérieur** de votre ordinateur de développement).

IV.3.3. COMPILATION ET LINKAGE DU SOURCE

Ce fichier est ensuite fourni au compilateur qui le **COMPILE** pour obtenir un fichier dit **OBJET** propre au microprocesseur considéré.

Il faut ensuite, dans certains cas, utiliser un autre programme appelé le **LINKER** (ou **EDITEUR DE LIENS**) qui va **particulariser / adapter** votre programme objet en fonction de **l'architecture matérielle** de votre carte électronique (adresses et taille de la ROM., adresses et taille de la RAM ...) ... bref en fonction du **MAPPING** de la carte.

Après cette étape on obtient un fichier **EXECUTABLE** (**mais qui n'a pas encore été exécuté**) par le microprocesseur / microcontrôleur de votre carte électronique applicative.

IV.3.4. TELECHARGEMENT SUR LA CARTE APPLICATIVE

Cette méthode est applicable si vous disposez d'une carte munie d'une liaison avec votre ordinateur de développement et d'un **MONITEUR implanté sur une ROM** de votre carte.

Un **MONITEUR** est un petit programme qui est capable d'offrir à l'utilisateur un rudiment d'interface homme/machine (c'est-à-dire entre vous et votre carte applicative) :

- communication avec votre ordinateur de développement (clavier + écran),
- message de bienvenue (**PROMPT**),
- possibilité de visualisation / modification du contenu des mémoires de votre carte,
- possibilité d'affichage/modification des registres de votre microprocesseur / microcontrôleur,
- possibilité de lancer un programme en mode « temps réel » ou en pas à pas (mode TRACE),
- et enfin possibilité d'envoyer votre exécutable (issu de la phase **IV.3.3** précédente) sur la mémoire de votre carte applicative (qu'on appelle aussi pour cette raison la carte **CIBLE** ou la **TARGET**).

Exercice :

D'après ce qui vient d'être dit sur le téléchargement, dans quel type de mémoire de la carte applicative télécharge t'on l'exécutable ?

IV.3.5. TESTS ET MISE AU POINT

Grâce au moniteur, on commence alors les tests du programme sur la carte cible.

Si ça marche du premier coup, **vous êtes un « as »** et pouvez passer à l'étape suivante.

Sinon, et c'est souvent le cas, il faut remonter à la phase **IV.3.2** (ou **IV.3.1** éventuellement), corriger les erreurs sur son source, recompiler/linker, retélécharger et recommencer les tests

C'est la phase de mise au point qui peut parfois être longue ... et fastidieuse, mais qui demande de la rigueur, et la méthodologie ... et souvent de l'acharnement (avec surtout une forte envie d'y arriver).

IV.3.6. IMPLANTATION EN ROM

Vous pouvez alors implanter votre exécutable validé sur une ROM de votre carte cible.

A partir du fichier exécutable précédent, on utilise un appareil appelé " **programmeur ou claqueur de R.O.M.** "

On programme ainsi une R.O.M., contenant l'exécutable qu'il ne reste plus qu'à placer sur le support que vous aurez prévu pour elle sur votre carte électronique.

Remarque :

En toute rigueur, il s'agira :

- soit d'une ROM de type **E.P.R.O.M.** (**Erasable Programmable Read Only Memory**) qui possède la propriété d'être programmable des centaines de fois (elle est effaçable en la plaçant pendant un certain temps sous un rayonnement Ultra Violet et reprogrammable ensuite).
- soit d'une ROM de type **E.E.P.R.O.M.** (**Erasable Electrically Programmable Read Only Memory**) qui possède la propriété d'être programmable et effaçable électriquement (comme celles qui existent sur la carte cible qu'on utilisera en Travaux Pratiques).

A partir de là, plus besoin de votre ordinateur pour faire fonctionner votre carte cible :

le « cordon ombilical » a été coupé et votre application est autonome. Bravo !!!

V. QUELQUES CONSEILS PRATIQUES ET UTILES

La première fois qu'on se penche sur une carte à base de microprocesseur / microcontrôleur pour la mettre au point (ou la dépanner), on est un peu dérouté devant le nombre de composants et, il faut bien le dire, on ne sait pas par où commencer.

Pour avoir conçu, développé et testé plusieurs systèmes complets, voilà quelques conseils pratiques que je peux donner à un étudiant débutant mais soucieux de bien faire :

- **vérification de l'alimentation** : c'est tout bête, mais une petite L.E.D. placée sur la (ou les) alimentation(s) sera pratique et permet de voir d'un coup d'œil la présence ou l'absence des tensions. Pour des problèmes plus " pointus " où l'intégrité de l'alimentation est suspectée (parasites, composants qui chauffent ...), ne pas hésiter à utiliser l'oscilloscope, seul outil capable de visualiser plus finement l'allure réelle de la tension. Ne pas oublier de se placer en position **DC** pour voir la totalité de l'alimentation et en position **AC** pour la composante variable (parasites ...).

Beaucoup de dysfonctionnements peuvent venir d'un problème d'alimentation.

- **vérification du quartz et des horloges** : toujours à l'oscilloscope, visualiser les horloges du système (niveaux de tension, fréquences ...),

De même, beaucoup de dysfonctionnements peuvent venir d'un problème sur les horloges.

- pour **vérifier un échange de données entre le microprocesseur et la mémoire externe ou un circuit périphérique**, on synchronisera avantagusement son oscilloscope sur le $\overline{CS}$ de cette mémoire externe ou de ce circuit périphérique et on vérifiera les niveaux de tension sur les différents bus. L'utilisation d'un **analyseur logique** peut faciliter le travail puisqu'il permet de visualiser les bus en entier, de mettre en mémoire les chronogrammes pour les examiner et les analyser ensuite plus tranquillement, de se synchroniser sur telle ou telle combinaison présente sur un bus ...

- lorsque la sortie d'un circuit intégré est à zéro alors qu'elle devrait être théoriquement au niveau haut, cela ne veut pas forcément dire que ce composant est défectueux. Pour affirmer cela, il faut enlever tous autres circuits branchés sur cette sortie (sortie à vide),

- de façon générale, essayer de tester la carte fonction par fonction (démarche **rationnelle et scientifique**) : ce sera toujours plus simple que la carte complète. On reconnectera ensuite les fonctions entre elles lorsqu'elles auront été **vérifiées et validées individuellement**,

- on essaiera autant que possible de faire tourner des bouts de programmes **le plus simple possible** (boucles très courtes, voire nulles c'est-à-dire « rebouclant sur elles-mêmes »),

- et bien sûr les précautions d'usage, qu'on peut oublier quand on est concentré sur la résolution d'un **BUG** (erreur ...) :

pensez à couper l'alimentation lorsque vous enfoncez (ou retirez) un circuit intégré sur son support ou une carte électronique de son connecteur ...

Ces quelques conseils (et bien d'autres que vous apprendrez en Travaux Pratiques) devraient vous aider si vous conduisez un jour un projet dans ce type d'environnement.